

3_regression_project

September 2, 2026

1 Going Deeper into a regression model with a project

I hope you remember the Deep learning life cycle.

First step is loading the data.

1.1 Loading the data

I'll be using the Medical cost personal dataset from the [kaggle](#).

This dataset contains personal health insurance costs for 1338 individuals. The columns are as follows:

- age: age of primary beneficiary
- sex: insurance contractor gender, female, male
- bmi: Body mass index, providing an understanding of body fatness
- children: Number of children covered by health insurance / Number of dependents
- smoker: Smoking, yes or no
- region: the beneficiary's geographic information, Southeast, Southwest, Northeast, Northwest
- charges: Individual medical costs billed by health insurance

A small and nice data set to clean and do some deep learning experiments.

You can download the dataset from kaggle or you can directly load from github without any downloading.

Pandas can load raw data from links. Here's the [link](#)

```
[1]: import pandas as pd

data = pd.read_csv(
    "https://raw.githubusercontent.com/RishatTalukder/
↳Deep_Learning_With_Tensorflow/refs/heads/main/2_regression/insurance.csv"
)
data.head()
```

```
[1]:   age    sex    bmi  children  smoker    region    charges
0   19  female  27.900         0     yes  southwest  16884.92400
1   18   male  33.770         1     no   southeast   1725.55230
2   28   male  33.000         3     no   southeast   4449.46200
3   33   male  22.705         0     no  northwest  21984.47061
```

```
4    32    male  28.880          0    no  northwest   3866.85520
```

See? nice right?

We can now use this dataset.

So, time for the second step.

1.2 Data Analysis

First let's check the information of the dataset.

```
[2]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   str
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   str
5   region      1338 non-null   str
6   charges     1338 non-null   float64
dtypes: float64(2), int64(2), str(3)
memory usage: 94.5 KB
```

We have a total of 1338 rows and 7 columns and no missing values.

That makes our work simpler and I'm not thinking of going too deep, I'm going to keep it direct.

There's might be some categorical features in this dataset. Let's check that.

```
[3]: data.describe()
```

```
[3]:
```

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

Only `age`, `bmi`, `children` and `charges` are numerical features.

So, the rest can be categorical. Let's see..

```
[4]: data.smoker.unique()
```

```
[4]: <ArrowStringArray>  
      ['yes', 'no']  
      Length: 2, dtype: str
```

```
[5]: data.sex.unique()
```

```
[5]: <ArrowStringArray>  
      ['female', 'male']  
      Length: 2, dtype: str
```

```
[6]: data.region.unique()
```

```
[6]: <ArrowStringArray>  
      ['southwest', 'southeast', 'northwest', 'northeast']  
      Length: 4, dtype: str
```

Yeah, looks like `smoker`, `sex` and `region` features are categorical features.

There's another feature that can be categorical even if it is numerical.

```
[7]: data.children.unique()
```

```
[7]: array([0, 1, 3, 2, 5, 4])
```

Yeap, as you can see the children column can be categorical. But as this features represents the number of children, I'm going to keep it numerical.

Now, we have to transform the categorical feature to numerical feature. Because these feature are in string format.

So, now we can **one hot encode** the categorical features.

You can learn **One-hot encoding** from [here](#)

A little summery of **One-hot encoding**:

- One-hot encoding is a technique used to convert categorical variables into numerical variables.
- It creates a new column for each unique category in the categorical variable.
- Each cell in the new column is either 0 or 1, indicating whether the corresponding category is present in the original categorical variable.
- The value 1 indicates the presence of the category, while 0 indicates its absence.
- Let's say we have a categorical column like this:

sex
male
male

sex
female
female
male

- we takes all the unique values of the column and create a new column for each unique value.

sex_male	sex_female
1	0
1	0
0	1
0	1
1	0

- Each of these columns have a value of 1 or 0, indicating whether the corresponding category is present in the original categorical variable.
- Occasionally, we can drop one of these columns, because in this case if a row has 1 in the `sex_male` column, it means that the row has the `male` category, there is no chance that the row can have the `female` category. So, we can drop the `sex_female` column. This logic can be applied to any number of categories and we can drop a column and still the ammount of information will stay the same.

So, how do we do this?

We can Either use a `pandas dataframe` method or a `sklearn preprocessing class` to do this.

I prefer to use both, which ever is convenient for me.

I show you both.

1.2.1 One hot encoding using `get_dummies()`

```
[8]: pd.get_dummies(
      data,
      columns=["sex", "smoker", "region"],
      drop_first=True,
    )
```

```
[8]:   age    bmi  children    charges  sex_male  smoker_yes  \
0    19  27.900         0  16884.92400    False         True
1    18  33.770         1   1725.55230     True         False
2    28  33.000         3   4449.46200     True         False
3    33  22.705         0  21984.47061     True         False
4    32  28.880         0   3866.85520     True         False
...  ...    ...      ...      ...      ...      ...
1333  50  30.970         3  10600.54830     True         False
1334  18  31.920         0   2205.98080    False         False
1335  18  36.850         0   1629.83350    False         False
```

1336	21	25.800	0	2007.94500	False	False
1337	61	29.070	0	29141.36030	False	True

	region_northwest	region_southeast	region_southwest
0	False	False	True
1	False	True	False
2	False	True	False
3	True	False	False
4	True	False	False
...	...	...	...
1333	True	False	False
1334	False	False	False
1335	False	True	False
1336	False	False	True
1337	True	False	False

[1338 rows x 9 columns]

`pd.get_dummies` is a function that takes a pandas dataframe as input and returns a new dataframe with one-hot encoded columns for the specified categorical columns.

You can specify which columns to one-hot encode using the `columns` parameter or if you dont it'll one-hot encode all column with object, string or category dtype.

If you want to drop one of the one-hot encoded columns, you can use the `drop_first` parameter.

We can do the same using `sklearn OneHotEncoder` class.

```
[9]: from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    sparse_output=False,
    drop="first",
)
encoder.set_output(transform="pandas")
encoder.fit_transform(data[["sex", "smoker", "region"]])
```

[9]:	sex_male	smoker_yes	region_northwest	region_southeast	\
0	0.0	1.0	0.0	0.0	
1	1.0	0.0	0.0	1.0	
2	1.0	0.0	0.0	1.0	
3	1.0	0.0	1.0	0.0	
4	1.0	0.0	1.0	0.0	
...	...	...	...	...	
1333	1.0	0.0	1.0	0.0	
1334	0.0	0.0	0.0	0.0	
1335	0.0	0.0	0.0	1.0	
1336	0.0	0.0	0.0	0.0	
1337	0.0	1.0	1.0	0.0	

```

        region_southwest
0          1.0
1          0.0
2          0.0
3          0.0
4          0.0
...
1333        0.0
1334        0.0
1335        0.0
1336        1.0
1337        0.0

```

[1338 rows x 5 columns]

Here I'm using `OneHotEncoder` class from `sklearn` to do the same thing.

Inside, I've set `sparse_output=False` because I want the output to be a pandas dataframe and not a sparse matrix.

And I've set `drop="first"` because I want to drop one of the one-hot encoded columns.

I've set `transform="pandas"` in using the `set_output` method because I want the output to be a pandas dataframe.

And after that I'm using `fit_transform` method to fit the encoder to the data and transform the data.

And we have the one-hot encoded dataframe.

I'll set up a pipeline for this process in this article later. But for now let's stick with `get_dummies` method.

```
[10]: data = pd.get_dummies(
        data,
        columns=["sex", "smoker", "region"],
        drop_first=True,
    )
```

```
[11]: data.head()
```

```
[11]:
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest \
0	19	27.900	0	16884.92400	False	True	False
1	18	33.770	1	1725.55230	True	False	False
2	28	33.000	3	4449.46200	True	False	False
3	33	22.705	0	21984.47061	True	False	True
4	32	28.880	0	3866.85520	True	False	True

```

        region_southeast  region_southwest

```

0	False	True
1	True	False
2	True	False
3	False	False
4	False	False

Nice.

Now, we can just train a model. But first we have to split.

1.3 Splitting the data

In this data set the target is to predict the **charges** column which represents the individual medical costs billed by health insurance.

```
[12]: X = data.drop(
        columns=["charges"],
    )

    y = data["charges"]
```

Now we split.

```
[13]: from sklearn.model_selection import train_test_split

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
    )
```

```
[14]: X_train.shape
```

```
[14]: (1070, 8)
```

And we have our train and test sets.

One thing left to do.

1.4 Building the model and training it

I'll make a very small model for the first try.

```
[15]: import tensorflow as tf
    from keras import (
        layers,
        losses,
        metrics,
        models,
```

```
optimizers,  
)
```

2026-09-02 15:37:44.435576: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

```
[16]: model = models.Sequential([  
    layers.Input(shape=(X_train.shape[1],)),  
    layers.Dense(10, activation="relu"),  
    layers.Dense(5, activation="relu"),  
    layers.Dense(1),  
)  
  
model.compile(  
    optimizer=optimizers.Adam(),  
    loss=losses.MeanAbsoluteError(),  
    metrics=[metrics.RootMeanSquaredError()],  
)
```

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1788341879.544891 12726 gpu_device.cc:2020] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 9552 MB memory: -> device: 0, name: NVIDIA GeForce RTX 3060, pci bus id: 0000:05:00.0, compute capability: 8.6

This model will have 3 layers, the first one will have 10 nodes, the second one will have 5 nodes and the last one will have 1 node.

And I'm using Adam optimizer and MeanAbsoluteError loss function and RootMeanSquaredError metric to see if the models are overfitting or underfitting.

```
[17]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	90
dense_1 (Dense)	(None, 5)	55
dense_2 (Dense)	(None, 1)	6

Total params: 151 (604.00 B)

Trainable params: 151 (604.00 B)

Non-trainable params: 0 (0.00 B)

Let's see how our model performs.

```
[18]: model.fit(  
        X_train,  
        y_train,  
        epochs=100  
    )
```

Epoch 1/100

2026-09-02 15:38:03.814769: I external/local_xla/xla/service/service.cc:163] XLA service 0x77f8c4006790 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:

2026-09-02 15:38:03.814783: I external/local_xla/xla/service/service.cc:171] StreamExecutor device (0): NVIDIA GeForce RTX 3060, Compute Capability 8.6

2026-09-02 15:38:03.990849: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var `MLIR\_CRASH\_REPRODUCER\_DIRECTORY` to enable.

2026-09-02 15:38:04.420435: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 91801

1/34 1:24 3s/step - loss:
16697.7266 - root_mean_squared_error: 23484.3223

I0000 00:00:1788341885.920690 13237 device_compiler.h:196] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

34/34 3s 21ms/step -
loss: 13344.0449 - root_mean_squared_error: 17955.3398

Epoch 2/100

34/34 0s 1ms/step - loss:
13341.7197 - root_mean_squared_error: 17953.5293

Epoch 3/100

34/34 0s 1ms/step - loss:
13339.0830 - root_mean_squared_error: 17951.4473

Epoch 4/100

34/34 0s 1ms/step - loss:
13335.9189 - root_mean_squared_error: 17948.9375

Epoch 5/100

34/34 0s 1ms/step - loss:
13332.0898 - root_mean_squared_error: 17945.9395

Epoch 6/100
34/34 0s 1ms/step - loss:
13327.3477 - root_mean_squared_error: 17942.1738
Epoch 7/100
34/34 0s 1ms/step - loss:
13321.3623 - root_mean_squared_error: 17937.4824
Epoch 8/100
34/34 0s 1ms/step - loss:
13313.7744 - root_mean_squared_error: 17931.4180
Epoch 9/100
34/34 0s 1ms/step - loss:
13304.2324 - root_mean_squared_error: 17923.9258
Epoch 10/100
34/34 0s 1ms/step - loss:
13292.3398 - root_mean_squared_error: 17914.3789
Epoch 11/100
34/34 0s 1ms/step - loss:
13277.7656 - root_mean_squared_error: 17903.1172
Epoch 12/100
34/34 0s 1ms/step - loss:
13259.9238 - root_mean_squared_error: 17888.8828
Epoch 13/100
34/34 0s 1ms/step - loss:
13237.7295 - root_mean_squared_error: 17871.4434
Epoch 14/100
34/34 0s 1ms/step - loss:
13210.1963 - root_mean_squared_error: 17849.5273
Epoch 15/100
34/34 0s 1ms/step - loss:
13176.8203 - root_mean_squared_error: 17823.2637
Epoch 16/100
34/34 0s 1ms/step - loss:
13137.2979 - root_mean_squared_error: 17792.1855
Epoch 17/100
34/34 0s 1ms/step - loss:
13091.4111 - root_mean_squared_error: 17755.5312
Epoch 18/100
34/34 0s 1ms/step - loss:
13038.7969 - root_mean_squared_error: 17714.7129
Epoch 19/100
34/34 0s 1ms/step - loss:
12979.2402 - root_mean_squared_error: 17667.9590
Epoch 20/100
34/34 0s 1ms/step - loss:
12912.3369 - root_mean_squared_error: 17615.3711
Epoch 21/100
34/34 0s 1ms/step - loss:
12838.0518 - root_mean_squared_error: 17556.5625

Epoch 22/100
34/34 0s 1ms/step - loss:
12756.0244 - root_mean_squared_error: 17492.4863
Epoch 23/100
34/34 0s 1ms/step - loss:
12666.0762 - root_mean_squared_error: 17424.3652
Epoch 24/100
34/34 0s 1ms/step - loss:
12567.8193 - root_mean_squared_error: 17347.6191
Epoch 25/100
34/34 0s 1ms/step - loss:
12460.8848 - root_mean_squared_error: 17263.5137
Epoch 26/100
34/34 0s 1ms/step - loss:
12345.6494 - root_mean_squared_error: 17175.7559
Epoch 27/100
34/34 0s 1ms/step - loss:
12221.4863 - root_mean_squared_error: 17081.5430
Epoch 28/100
34/34 0s 1ms/step - loss:
12087.8525 - root_mean_squared_error: 16976.6074
Epoch 29/100
34/34 0s 1ms/step - loss:
11946.1074 - root_mean_squared_error: 16868.2539
Epoch 30/100
34/34 0s 1ms/step - loss:
11794.6621 - root_mean_squared_error: 16754.7109
Epoch 31/100
34/34 0s 1ms/step - loss:
11636.3545 - root_mean_squared_error: 16634.5664
Epoch 32/100
34/34 0s 1ms/step - loss:
11469.6494 - root_mean_squared_error: 16509.2852
Epoch 33/100
34/34 0s 1ms/step - loss:
11294.2275 - root_mean_squared_error: 16375.5986
Epoch 34/100
34/34 0s 1ms/step - loss:
11113.6846 - root_mean_squared_error: 16239.9482
Epoch 35/100
34/34 0s 1ms/step - loss:
10928.5693 - root_mean_squared_error: 16095.7500
Epoch 36/100
34/34 0s 1ms/step - loss:
10740.4199 - root_mean_squared_error: 15951.1045
Epoch 37/100
34/34 0s 1ms/step - loss:
10551.1396 - root_mean_squared_error: 15800.3369

Epoch 38/100
34/34 0s 1ms/step - loss:
10361.3379 - root_mean_squared_error: 15649.8584
Epoch 39/100
34/34 0s 2ms/step - loss:
10172.1113 - root_mean_squared_error: 15495.1240
Epoch 40/100
34/34 0s 2ms/step - loss:
9983.0928 - root_mean_squared_error: 15335.7930
Epoch 41/100
34/34 0s 1ms/step - loss:
9797.8428 - root_mean_squared_error: 15184.8008
Epoch 42/100
34/34 0s 1ms/step - loss:
9611.9551 - root_mean_squared_error: 15027.0957
Epoch 43/100
34/34 0s 1ms/step - loss:
9428.0889 - root_mean_squared_error: 14863.5186
Epoch 44/100
34/34 0s 1ms/step - loss:
9244.6104 - root_mean_squared_error: 14704.1396
Epoch 45/100
34/34 0s 1ms/step - loss:
9061.6680 - root_mean_squared_error: 14543.4766
Epoch 46/100
34/34 0s 1ms/step - loss:
8891.2969 - root_mean_squared_error: 14385.4150
Epoch 47/100
34/34 0s 1ms/step - loss:
8727.9023 - root_mean_squared_error: 14230.4092
Epoch 48/100
34/34 0s 1ms/step - loss:
8572.3359 - root_mean_squared_error: 14081.3809
Epoch 49/100
34/34 0s 1ms/step - loss:
8424.4258 - root_mean_squared_error: 13931.5527
Epoch 50/100
34/34 0s 1ms/step - loss:
8286.8359 - root_mean_squared_error: 13789.9150
Epoch 51/100
34/34 0s 1ms/step - loss:
8160.1475 - root_mean_squared_error: 13653.9580
Epoch 52/100
34/34 0s 1ms/step - loss:
8041.4404 - root_mean_squared_error: 13523.7773
Epoch 53/100
34/34 0s 1ms/step - loss:
7936.0615 - root_mean_squared_error: 13395.0713

Epoch 54/100
34/34 0s 1ms/step - loss:
7843.3325 - root_mean_squared_error: 13280.8223
Epoch 55/100
34/34 0s 1ms/step - loss:
7765.5513 - root_mean_squared_error: 13171.2324
Epoch 56/100
34/34 0s 1ms/step - loss:
7698.8354 - root_mean_squared_error: 13072.3008
Epoch 57/100
34/34 0s 1ms/step - loss:
7640.5337 - root_mean_squared_error: 12982.2930
Epoch 58/100
34/34 0s 1ms/step - loss:
7596.7368 - root_mean_squared_error: 12901.5039
Epoch 59/100
34/34 0s 1ms/step - loss:
7559.0859 - root_mean_squared_error: 12829.4258
Epoch 60/100
34/34 0s 1ms/step - loss:
7529.6323 - root_mean_squared_error: 12768.0469
Epoch 61/100
34/34 0s 1ms/step - loss:
7507.5620 - root_mean_squared_error: 12710.7314
Epoch 62/100
34/34 0s 1ms/step - loss:
7489.1021 - root_mean_squared_error: 12661.5723
Epoch 63/100
34/34 0s 1ms/step - loss:
7476.0562 - root_mean_squared_error: 12614.6406
Epoch 64/100
34/34 0s 1ms/step - loss:
7464.9995 - root_mean_squared_error: 12582.2061
Epoch 65/100
34/34 0s 1ms/step - loss:
7456.1553 - root_mean_squared_error: 12553.2148
Epoch 66/100
34/34 0s 1ms/step - loss:
7447.9961 - root_mean_squared_error: 12519.9541
Epoch 67/100
34/34 0s 1ms/step - loss:
7440.4878 - root_mean_squared_error: 12491.6836
Epoch 68/100
34/34 0s 1ms/step - loss:
7434.4751 - root_mean_squared_error: 12468.6650
Epoch 69/100
34/34 0s 1ms/step - loss:
7429.1113 - root_mean_squared_error: 12443.3838

Epoch 70/100
34/34 0s 1ms/step - loss:
7425.0547 - root_mean_squared_error: 12425.0879
Epoch 71/100
34/34 0s 1ms/step - loss:
7421.9790 - root_mean_squared_error: 12411.5947
Epoch 72/100
34/34 0s 1ms/step - loss:
7419.4507 - root_mean_squared_error: 12403.7666
Epoch 73/100
34/34 0s 1ms/step - loss:
7417.4741 - root_mean_squared_error: 12392.8838
Epoch 74/100
34/34 0s 1ms/step - loss:
7415.0605 - root_mean_squared_error: 12390.2109
Epoch 75/100
34/34 0s 1ms/step - loss:
7412.7959 - root_mean_squared_error: 12383.0820
Epoch 76/100
34/34 0s 1ms/step - loss:
7410.6411 - root_mean_squared_error: 12386.5674
Epoch 77/100
34/34 0s 1ms/step - loss:
7408.4707 - root_mean_squared_error: 12386.9150
Epoch 78/100
34/34 0s 1ms/step - loss:
7406.4102 - root_mean_squared_error: 12376.1855
Epoch 79/100
34/34 0s 1ms/step - loss:
7404.1489 - root_mean_squared_error: 12377.5146
Epoch 80/100
34/34 0s 1ms/step - loss:
7401.9390 - root_mean_squared_error: 12378.2832
Epoch 81/100
34/34 0s 1ms/step - loss:
7399.6221 - root_mean_squared_error: 12373.9678
Epoch 82/100
34/34 0s 1ms/step - loss:
7397.5347 - root_mean_squared_error: 12370.3457
Epoch 83/100
34/34 0s 1ms/step - loss:
7395.1738 - root_mean_squared_error: 12361.5098
Epoch 84/100
34/34 0s 1ms/step - loss:
7392.8594 - root_mean_squared_error: 12363.9961
Epoch 85/100
34/34 0s 1ms/step - loss:
7390.9668 - root_mean_squared_error: 12363.4639

```
Epoch 86/100
34/34          0s 1ms/step - loss:
7388.6245 - root_mean_squared_error: 12364.0615
Epoch 87/100
34/34          0s 1ms/step - loss:
7386.3911 - root_mean_squared_error: 12369.7598
Epoch 88/100
34/34          0s 1ms/step - loss:
7383.9844 - root_mean_squared_error: 12367.3037
Epoch 89/100
34/34          0s 1ms/step - loss:
7381.7705 - root_mean_squared_error: 12364.3359
Epoch 90/100
34/34          0s 1ms/step - loss:
7379.9443 - root_mean_squared_error: 12356.9971
Epoch 91/100
34/34          0s 1ms/step - loss:
7377.3745 - root_mean_squared_error: 12362.7461
Epoch 92/100
34/34          0s 1ms/step - loss:
7374.7979 - root_mean_squared_error: 12364.5342
Epoch 93/100
34/34          0s 1ms/step - loss:
7372.6289 - root_mean_squared_error: 12359.5928
Epoch 94/100
34/34          0s 1ms/step - loss:
7370.1289 - root_mean_squared_error: 12359.3066
Epoch 95/100
34/34          0s 1ms/step - loss:
7368.1494 - root_mean_squared_error: 12361.0811
Epoch 96/100
34/34          0s 1ms/step - loss:
7365.4727 - root_mean_squared_error: 12361.3613
Epoch 97/100
34/34          0s 1ms/step - loss:
7363.4658 - root_mean_squared_error: 12361.5635
Epoch 98/100
34/34          0s 1ms/step - loss:
7361.1406 - root_mean_squared_error: 12351.3008
Epoch 99/100
34/34          0s 1ms/step - loss:
7358.5015 - root_mean_squared_error: 12348.0527
Epoch 100/100
34/34          0s 1ms/step - loss:
7356.1377 - root_mean_squared_error: 12347.9775
```

[18]: <keras.src.callbacks.history.History at 0x77f9ec3a3390>

Alright, no good. The loss is still pretty High and the RMSE shows that the model is predicting some pretty bad values.

But let's just evaluate it on the testing data first.

```
[19]: model.evaluate(X_test, y_test)

9/9          0s 30ms/step - loss:
7491.6353 - root_mean_squared_error: 12483.1523
```

```
[19]: [7491.63525390625, 12483.15234375]
```

Doesn't seem so bad right. But If we see that mean and median of the target variable we might know how bad this model is.

```
[20]: y.mean(), y.median()

[20]: (np.float64(13270.422265141257), 9382.033)
```

The mean absolute error is the average of the absolute difference between the predicted and actual values.

So, on average the model is off by 7014 dollars whereas the median is 13270 dollars. This is bad.

And if you take the median it's even worse.

So, we need to improve the model.

How do we do that.

We can do:

- Make the model more complex
- Train for longer.
- Try different optimizers.

Let's start with making the model more complex.

1.4.1 Making the model more complex

This time let's go for 3 hidden layers and 64, 32, 16 neurons respectively.

```
[21]: model2 = models.Sequential(
    [
        layers.Input(shape=(X_train.shape[1],)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1),
    ]
)

model2.compile(
```



```

optimizer=optimizers.Adam(),
loss=losses.MeanAbsoluteError(),
metrics=[metrics.RootMeanSquaredError()],
)

model2.fit(
    X_train,
    y_train,
    epochs=100
)

```

```

Epoch 1/100
34/34          2s 27ms/step -
loss: 13339.8438 - root_mean_squared_error: 17951.7363
Epoch 2/100
34/34          0s 1ms/step - loss:
13309.0166 - root_mean_squared_error: 17927.2070
Epoch 3/100
34/34          0s 1ms/step - loss:
13180.5576 - root_mean_squared_error: 17825.2891
Epoch 4/100
34/34          0s 1ms/step - loss:
12678.2695 - root_mean_squared_error: 17430.3301
Epoch 5/100
34/34          0s 1ms/step - loss:
11242.0811 - root_mean_squared_error: 16346.7217
Epoch 6/100
34/34          0s 1ms/step - loss:
8759.0635 - root_mean_squared_error: 14297.3154
Epoch 7/100
34/34          0s 1ms/step - loss:
7454.9946 - root_mean_squared_error: 12492.3838
Epoch 8/100
34/34          0s 1ms/step - loss:
7394.9966 - root_mean_squared_error: 12332.7754
Epoch 9/100
34/34          0s 1ms/step - loss:
7372.7007 - root_mean_squared_error: 12351.2119
Epoch 10/100
34/34          0s 1ms/step - loss:
7347.6885 - root_mean_squared_error: 12390.7295
Epoch 11/100
34/34          0s 1ms/step - loss:
7316.3569 - root_mean_squared_error: 12398.3594
Epoch 12/100
34/34          0s 1ms/step - loss:
7291.1646 - root_mean_squared_error: 12360.0537

```

Epoch 13/100
34/34 0s 1ms/step - loss:
7265.8813 - root_mean_squared_error: 12350.4053
Epoch 14/100
34/34 0s 1ms/step - loss:
7244.1133 - root_mean_squared_error: 12435.1885
Epoch 15/100
34/34 0s 1ms/step - loss:
7216.0024 - root_mean_squared_error: 12340.5791
Epoch 16/100
34/34 0s 1ms/step - loss:
7187.1714 - root_mean_squared_error: 12417.0283
Epoch 17/100
34/34 0s 1ms/step - loss:
7163.1719 - root_mean_squared_error: 12374.9111
Epoch 18/100
34/34 0s 1ms/step - loss:
7135.1426 - root_mean_squared_error: 12410.1768
Epoch 19/100
34/34 0s 1ms/step - loss:
7106.9282 - root_mean_squared_error: 12418.0195
Epoch 20/100
34/34 0s 1ms/step - loss:
7079.4204 - root_mean_squared_error: 12433.3428
Epoch 21/100
34/34 0s 1ms/step - loss:
7052.7974 - root_mean_squared_error: 12425.1670
Epoch 22/100
34/34 0s 1ms/step - loss:
7027.8423 - root_mean_squared_error: 12455.3877
Epoch 23/100
34/34 0s 1ms/step - loss:
6992.2441 - root_mean_squared_error: 12463.5605
Epoch 24/100
34/34 0s 1ms/step - loss:
6963.2510 - root_mean_squared_error: 12510.2305
Epoch 25/100
34/34 0s 1ms/step - loss:
6933.4399 - root_mean_squared_error: 12464.7676
Epoch 26/100
34/34 0s 1ms/step - loss:
6906.1284 - root_mean_squared_error: 12539.8770
Epoch 27/100
34/34 0s 1ms/step - loss:
6874.5298 - root_mean_squared_error: 12543.4307
Epoch 28/100
34/34 0s 1ms/step - loss:
6848.2192 - root_mean_squared_error: 12513.8838

Epoch 29/100
34/34 0s 1ms/step - loss:
6808.9624 - root_mean_squared_error: 12599.7441
Epoch 30/100
34/34 0s 1ms/step - loss:
6777.8379 - root_mean_squared_error: 12610.6123
Epoch 31/100
34/34 0s 1ms/step - loss:
6747.8047 - root_mean_squared_error: 12594.0283
Epoch 32/100
34/34 0s 1ms/step - loss:
6728.0503 - root_mean_squared_error: 12648.0127
Epoch 33/100
34/34 0s 2ms/step - loss:
6693.7896 - root_mean_squared_error: 12656.4268
Epoch 34/100
34/34 0s 1ms/step - loss:
6666.1377 - root_mean_squared_error: 12651.4023
Epoch 35/100
34/34 0s 1ms/step - loss:
6640.2417 - root_mean_squared_error: 12631.7744
Epoch 36/100
34/34 0s 1ms/step - loss:
6625.8027 - root_mean_squared_error: 12650.8857
Epoch 37/100
34/34 0s 2ms/step - loss:
6608.3999 - root_mean_squared_error: 12665.3193
Epoch 38/100
34/34 0s 1ms/step - loss:
6595.0308 - root_mean_squared_error: 12678.3174
Epoch 39/100
34/34 0s 1ms/step - loss:
6579.9688 - root_mean_squared_error: 12686.7900
Epoch 40/100
34/34 0s 1ms/step - loss:
6568.5166 - root_mean_squared_error: 12655.9326
Epoch 41/100
34/34 0s 1ms/step - loss:
6561.6494 - root_mean_squared_error: 12657.0703
Epoch 42/100
34/34 0s 1ms/step - loss:
6548.1597 - root_mean_squared_error: 12636.7256
Epoch 43/100
34/34 0s 1ms/step - loss:
6541.4404 - root_mean_squared_error: 12652.6543
Epoch 44/100
34/34 0s 1ms/step - loss:
6537.4741 - root_mean_squared_error: 12643.3936

Epoch 45/100
34/34 0s 1ms/step - loss:
6525.1016 - root_mean_squared_error: 12644.6846
Epoch 46/100
34/34 0s 1ms/step - loss:
6513.7554 - root_mean_squared_error: 12607.7676
Epoch 47/100
34/34 0s 1ms/step - loss:
6503.9702 - root_mean_squared_error: 12626.0596
Epoch 48/100
34/34 0s 2ms/step - loss:
6492.4561 - root_mean_squared_error: 12595.6328
Epoch 49/100
34/34 0s 2ms/step - loss:
6480.5972 - root_mean_squared_error: 12568.0059
Epoch 50/100
34/34 0s 1ms/step - loss:
6472.1968 - root_mean_squared_error: 12608.6387
Epoch 51/100
34/34 0s 1ms/step - loss:
6468.1113 - root_mean_squared_error: 12559.5293
Epoch 52/100
34/34 0s 1ms/step - loss:
6450.8911 - root_mean_squared_error: 12563.9727
Epoch 53/100
34/34 0s 1ms/step - loss:
6443.3608 - root_mean_squared_error: 12532.9580
Epoch 54/100
34/34 0s 1ms/step - loss:
6430.6343 - root_mean_squared_error: 12518.9346
Epoch 55/100
34/34 0s 1ms/step - loss:
6419.7339 - root_mean_squared_error: 12508.0547
Epoch 56/100
34/34 0s 1ms/step - loss:
6411.7832 - root_mean_squared_error: 12506.5166
Epoch 57/100
34/34 0s 1ms/step - loss:
6397.7534 - root_mean_squared_error: 12493.1348
Epoch 58/100
34/34 0s 1ms/step - loss:
6384.6631 - root_mean_squared_error: 12485.6680
Epoch 59/100
34/34 0s 1ms/step - loss:
6371.7134 - root_mean_squared_error: 12451.0430
Epoch 60/100
34/34 0s 1ms/step - loss:
6356.7700 - root_mean_squared_error: 12442.8379

Epoch 61/100
34/34 0s 1ms/step - loss:
6344.9438 - root_mean_squared_error: 12428.0947
Epoch 62/100
34/34 0s 1ms/step - loss:
6332.8652 - root_mean_squared_error: 12394.0166
Epoch 63/100
34/34 0s 1ms/step - loss:
6321.0093 - root_mean_squared_error: 12373.8955
Epoch 64/100
34/34 0s 1ms/step - loss:
6302.1758 - root_mean_squared_error: 12355.6670
Epoch 65/100
34/34 0s 1ms/step - loss:
6285.9785 - root_mean_squared_error: 12320.9404
Epoch 66/100
34/34 0s 1ms/step - loss:
6270.6724 - root_mean_squared_error: 12299.2852
Epoch 67/100
34/34 0s 1ms/step - loss:
6249.2500 - root_mean_squared_error: 12279.2617
Epoch 68/100
34/34 0s 1ms/step - loss:
6236.2544 - root_mean_squared_error: 12271.2617
Epoch 69/100
34/34 0s 1ms/step - loss:
6209.5015 - root_mean_squared_error: 12225.5889
Epoch 70/100
34/34 0s 1ms/step - loss:
6191.7588 - root_mean_squared_error: 12183.1689
Epoch 71/100
34/34 0s 1ms/step - loss:
6173.5391 - root_mean_squared_error: 12157.0977
Epoch 72/100
34/34 0s 1ms/step - loss:
6154.1060 - root_mean_squared_error: 12135.8164
Epoch 73/100
34/34 0s 1ms/step - loss:
6126.5923 - root_mean_squared_error: 12086.9258
Epoch 74/100
34/34 0s 1ms/step - loss:
6095.7998 - root_mean_squared_error: 12055.0928
Epoch 75/100
34/34 0s 1ms/step - loss:
6082.3496 - root_mean_squared_error: 11965.9463
Epoch 76/100
34/34 0s 1ms/step - loss:
6049.1582 - root_mean_squared_error: 11979.8418

Epoch 77/100
 34/34 0s 1ms/step - loss:
 6031.5146 - root_mean_squared_error: 11904.0674
 Epoch 78/100
 34/34 0s 1ms/step - loss:
 5989.5781 - root_mean_squared_error: 11833.1240
 Epoch 79/100
 34/34 0s 1ms/step - loss:
 5949.5151 - root_mean_squared_error: 11803.7256
 Epoch 80/100
 34/34 0s 1ms/step - loss:
 5912.1685 - root_mean_squared_error: 11740.2656
 Epoch 81/100
 34/34 0s 1ms/step - loss:
 5878.9512 - root_mean_squared_error: 11667.4170
 Epoch 82/100
 34/34 0s 1ms/step - loss:
 5835.7583 - root_mean_squared_error: 11604.0820
 Epoch 83/100
 34/34 0s 1ms/step - loss:
 5791.5688 - root_mean_squared_error: 11522.8281
 Epoch 84/100
 34/34 0s 1ms/step - loss:
 5744.2446 - root_mean_squared_error: 11442.5254
 Epoch 85/100
 34/34 0s 1ms/step - loss:
 5700.7207 - root_mean_squared_error: 11376.0889
 Epoch 86/100
 34/34 0s 1ms/step - loss:
 5644.7300 - root_mean_squared_error: 11256.4707
 Epoch 87/100
 34/34 0s 1ms/step - loss:
 5580.7754 - root_mean_squared_error: 11165.9873
 Epoch 88/100
 34/34 0s 1ms/step - loss:
 5508.5298 - root_mean_squared_error: 11045.9844
 Epoch 89/100
 34/34 0s 1ms/step - loss:
 5438.7935 - root_mean_squared_error: 10922.2197
 Epoch 90/100
 34/34 0s 1ms/step - loss:
 5357.9746 - root_mean_squared_error: 10804.9336
 Epoch 91/100
 34/34 0s 1ms/step - loss:
 5276.9473 - root_mean_squared_error: 10642.8027
 Epoch 92/100
 34/34 0s 1ms/step - loss:
 5174.1504 - root_mean_squared_error: 10477.4072

```

Epoch 93/100
34/34          0s 1ms/step - loss:
5059.8140 - root_mean_squared_error: 10308.0127
Epoch 94/100
34/34          0s 1ms/step - loss:
4938.9805 - root_mean_squared_error: 10121.2812
Epoch 95/100
34/34          0s 1ms/step - loss:
4811.2720 - root_mean_squared_error: 9882.5010
Epoch 96/100
34/34          0s 1ms/step - loss:
4677.7144 - root_mean_squared_error: 9629.8916
Epoch 97/100
34/34          0s 1ms/step - loss:
4530.7583 - root_mean_squared_error: 9409.5596
Epoch 98/100
34/34          0s 1ms/step - loss:
4379.9370 - root_mean_squared_error: 9156.3301
Epoch 99/100
34/34          0s 1ms/step - loss:
4248.4985 - root_mean_squared_error: 8914.8828
Epoch 100/100
34/34          0s 1ms/step - loss:
4131.3291 - root_mean_squared_error: 8673.6953

```

[21]: <keras.src.callbacks.history.History at 0x77f9e8271410>

Well, now it looks better than before. I'm sure it will perform better with testing set too...

But I want to see if it can get even better. I'll the fit method again for 100 epochs and The model will continue training from where it left off.

```

[22]: model2.fit(
        X_train,
        y_train,
        epochs=100
    )

```

```

Epoch 1/100
34/34          0s 1ms/step - loss:
4026.5317 - root_mean_squared_error: 8482.2510
Epoch 2/100
34/34          0s 1ms/step - loss:
3929.5945 - root_mean_squared_error: 8291.8564
Epoch 3/100
34/34          0s 1ms/step - loss:
3865.2478 - root_mean_squared_error: 8117.9902
Epoch 4/100
34/34          0s 1ms/step - loss:

```

```

3880.7698 - root_mean_squared_error: 8034.4229
Epoch 5/100
34/34          0s 1ms/step - loss:
3774.5281 - root_mean_squared_error: 7871.8335
Epoch 6/100
34/34          0s 1ms/step - loss:
3721.5916 - root_mean_squared_error: 7765.6494
Epoch 7/100
34/34          0s 1ms/step - loss:
3699.7976 - root_mean_squared_error: 7676.1201
Epoch 8/100
34/34          0s 1ms/step - loss:
3669.6711 - root_mean_squared_error: 7604.1597
Epoch 9/100
34/34          0s 1ms/step - loss:
3650.6406 - root_mean_squared_error: 7554.9932
Epoch 10/100
34/34          0s 1ms/step - loss:
3643.3865 - root_mean_squared_error: 7498.1772
Epoch 11/100
34/34          0s 1ms/step - loss:
3631.4570 - root_mean_squared_error: 7470.8193
Epoch 12/100
34/34          0s 1ms/step - loss:
3623.4651 - root_mean_squared_error: 7384.7393
Epoch 13/100
34/34          0s 2ms/step - loss:
3620.4883 - root_mean_squared_error: 7354.2515
Epoch 14/100
34/34          0s 1ms/step - loss:
3620.1375 - root_mean_squared_error: 7313.2661
Epoch 15/100
34/34          0s 1ms/step - loss:
3593.2197 - root_mean_squared_error: 7262.4478
Epoch 16/100
34/34          0s 1ms/step - loss:
3584.5229 - root_mean_squared_error: 7180.6069
Epoch 17/100
34/34          0s 1ms/step - loss:
3554.9402 - root_mean_squared_error: 7142.0771
Epoch 18/100
34/34          0s 1ms/step - loss:
3545.0571 - root_mean_squared_error: 7110.6743
Epoch 19/100
34/34          0s 1ms/step - loss:
3543.8823 - root_mean_squared_error: 7060.1333
Epoch 20/100
34/34          0s 1ms/step - loss:

```


3529.5320 - root_mean_squared_error: 7005.5493
Epoch 21/100
34/34 0s 1ms/step - loss:
3513.3831 - root_mean_squared_error: 6955.3345
Epoch 22/100
34/34 0s 1ms/step - loss:
3499.1274 - root_mean_squared_error: 6916.9766
Epoch 23/100
34/34 0s 1ms/step - loss:
3493.3672 - root_mean_squared_error: 6870.3472
Epoch 24/100
34/34 0s 1ms/step - loss:
3497.0618 - root_mean_squared_error: 6817.8110
Epoch 25/100
34/34 0s 1ms/step - loss:
3509.6711 - root_mean_squared_error: 6768.6572
Epoch 26/100
34/34 0s 1ms/step - loss:
3493.0278 - root_mean_squared_error: 6746.6255
Epoch 27/100
34/34 0s 1ms/step - loss:
3472.9185 - root_mean_squared_error: 6712.6904
Epoch 28/100
34/34 0s 1ms/step - loss:
3442.2695 - root_mean_squared_error: 6672.8315
Epoch 29/100
34/34 0s 1ms/step - loss:
3449.3118 - root_mean_squared_error: 6634.0054
Epoch 30/100
34/34 0s 1ms/step - loss:
3434.2578 - root_mean_squared_error: 6604.9575
Epoch 31/100
34/34 0s 1ms/step - loss:
3414.0552 - root_mean_squared_error: 6569.0771
Epoch 32/100
34/34 0s 1ms/step - loss:
3411.9004 - root_mean_squared_error: 6542.8271
Epoch 33/100
34/34 0s 1ms/step - loss:
3402.4573 - root_mean_squared_error: 6526.3882
Epoch 34/100
34/34 0s 1ms/step - loss:
3393.2405 - root_mean_squared_error: 6499.9058
Epoch 35/100
34/34 0s 1ms/step - loss:
3378.9307 - root_mean_squared_error: 6476.6558
Epoch 36/100
34/34 0s 1ms/step - loss:

```

3381.5376 - root_mean_squared_error: 6458.0430
Epoch 37/100
34/34          0s 1ms/step - loss:
3374.9790 - root_mean_squared_error: 6434.0605
Epoch 38/100
34/34          0s 1ms/step - loss:
3349.3945 - root_mean_squared_error: 6431.3833
Epoch 39/100
34/34          0s 1ms/step - loss:
3335.9470 - root_mean_squared_error: 6430.3008
Epoch 40/100
34/34          0s 1ms/step - loss:
3333.5864 - root_mean_squared_error: 6421.1025
Epoch 41/100
34/34          0s 1ms/step - loss:
3313.2856 - root_mean_squared_error: 6413.3105
Epoch 42/100
34/34          0s 1ms/step - loss:
3308.1260 - root_mean_squared_error: 6427.3315
Epoch 43/100
34/34          0s 1ms/step - loss:
3313.7854 - root_mean_squared_error: 6433.0610
Epoch 44/100
34/34          0s 1ms/step - loss:
3308.0776 - root_mean_squared_error: 6431.9536
Epoch 45/100
34/34          0s 1ms/step - loss:
3313.1941 - root_mean_squared_error: 6428.8838
Epoch 46/100
34/34          0s 2ms/step - loss:
3315.5054 - root_mean_squared_error: 6445.6353
Epoch 47/100
34/34          0s 1ms/step - loss:
3316.6055 - root_mean_squared_error: 6482.6470
Epoch 48/100
34/34          0s 1ms/step - loss:
3276.5930 - root_mean_squared_error: 6470.6401
Epoch 49/100
34/34          0s 1ms/step - loss:
3264.6101 - root_mean_squared_error: 6505.9531
Epoch 50/100
34/34          0s 1ms/step - loss:
3248.5044 - root_mean_squared_error: 6514.6191
Epoch 51/100
34/34          0s 1ms/step - loss:
3244.5220 - root_mean_squared_error: 6541.2432
Epoch 52/100
34/34          0s 1ms/step - loss:

```

3236.5188 - root_mean_squared_error: 6566.3213
Epoch 53/100
34/34 0s 1ms/step - loss:
3236.6643 - root_mean_squared_error: 6594.7695
Epoch 54/100
34/34 0s 1ms/step - loss:
3224.1355 - root_mean_squared_error: 6625.7246
Epoch 55/100
34/34 0s 1ms/step - loss:
3212.8379 - root_mean_squared_error: 6660.5103
Epoch 56/100
34/34 0s 1ms/step - loss:
3215.8435 - root_mean_squared_error: 6700.5322
Epoch 57/100
34/34 0s 1ms/step - loss:
3229.8552 - root_mean_squared_error: 6749.7588
Epoch 58/100
34/34 0s 1ms/step - loss:
3197.0828 - root_mean_squared_error: 6815.4072
Epoch 59/100
34/34 0s 1ms/step - loss:
3191.3782 - root_mean_squared_error: 6823.0454
Epoch 60/100
34/34 0s 1ms/step - loss:
3194.3235 - root_mean_squared_error: 6837.0791
Epoch 61/100
34/34 0s 1ms/step - loss:
3176.6741 - root_mean_squared_error: 6857.6548
Epoch 62/100
34/34 0s 1ms/step - loss:
3187.7852 - root_mean_squared_error: 6888.4771
Epoch 63/100
34/34 0s 1ms/step - loss:
3206.8279 - root_mean_squared_error: 6895.1274
Epoch 64/100
34/34 0s 1ms/step - loss:
3190.7961 - root_mean_squared_error: 6875.1763
Epoch 65/100
34/34 0s 1ms/step - loss:
3175.2903 - root_mean_squared_error: 6833.1538
Epoch 66/100
34/34 0s 1ms/step - loss:
3178.8584 - root_mean_squared_error: 6863.4648
Epoch 67/100
34/34 0s 1ms/step - loss:
3195.9614 - root_mean_squared_error: 6885.1177
Epoch 68/100
34/34 0s 1ms/step - loss:

3172.4355 - root_mean_squared_error: 6845.9858
Epoch 69/100
34/34 0s 1ms/step - loss:
3169.6748 - root_mean_squared_error: 6782.1973
Epoch 70/100
34/34 0s 1ms/step - loss:
3170.3882 - root_mean_squared_error: 6871.1377
Epoch 71/100
34/34 0s 1ms/step - loss:
3157.3477 - root_mean_squared_error: 6896.8394
Epoch 72/100
34/34 0s 1ms/step - loss:
3173.4563 - root_mean_squared_error: 6856.1099
Epoch 73/100
34/34 0s 1ms/step - loss:
3181.2991 - root_mean_squared_error: 6864.8193
Epoch 74/100
34/34 0s 1ms/step - loss:
3155.7566 - root_mean_squared_error: 6837.2563
Epoch 75/100
34/34 0s 1ms/step - loss:
3156.6670 - root_mean_squared_error: 6871.0200
Epoch 76/100
34/34 0s 1ms/step - loss:
3151.8567 - root_mean_squared_error: 6834.9038
Epoch 77/100
34/34 0s 1ms/step - loss:
3134.3523 - root_mean_squared_error: 6838.0698
Epoch 78/100
34/34 0s 1ms/step - loss:
3135.4468 - root_mean_squared_error: 6863.2290
Epoch 79/100
34/34 0s 1ms/step - loss:
3157.1250 - root_mean_squared_error: 6819.1147
Epoch 80/100
34/34 0s 1ms/step - loss:
3138.9614 - root_mean_squared_error: 6851.0220
Epoch 81/100
34/34 0s 1ms/step - loss:
3131.2156 - root_mean_squared_error: 6817.5508
Epoch 82/100
34/34 0s 1ms/step - loss:
3140.7256 - root_mean_squared_error: 6879.0659
Epoch 83/100
34/34 0s 1ms/step - loss:
3131.9500 - root_mean_squared_error: 6759.6792
Epoch 84/100
34/34 0s 1ms/step - loss:

3140.6169 - root_mean_squared_error: 6817.7451
Epoch 85/100
34/34 0s 1ms/step - loss:
3174.6340 - root_mean_squared_error: 6787.0928
Epoch 86/100
34/34 0s 1ms/step - loss:
3134.0200 - root_mean_squared_error: 6782.7666
Epoch 87/100
34/34 0s 1ms/step - loss:
3117.0098 - root_mean_squared_error: 6841.3076
Epoch 88/100
34/34 0s 1ms/step - loss:
3116.4309 - root_mean_squared_error: 6795.3784
Epoch 89/100
34/34 0s 1ms/step - loss:
3107.9441 - root_mean_squared_error: 6806.4214
Epoch 90/100
34/34 0s 1ms/step - loss:
3112.9773 - root_mean_squared_error: 6824.3379
Epoch 91/100
34/34 0s 1ms/step - loss:
3110.4534 - root_mean_squared_error: 6779.8281
Epoch 92/100
34/34 0s 1ms/step - loss:
3102.0410 - root_mean_squared_error: 6803.2349
Epoch 93/100
34/34 0s 1ms/step - loss:
3106.4580 - root_mean_squared_error: 6762.1621
Epoch 94/100
34/34 0s 1ms/step - loss:
3103.7388 - root_mean_squared_error: 6826.2695
Epoch 95/100
34/34 0s 1ms/step - loss:
3102.0107 - root_mean_squared_error: 6797.6660
Epoch 96/100
34/34 0s 1ms/step - loss:
3118.4531 - root_mean_squared_error: 6759.0303
Epoch 97/100
34/34 0s 1ms/step - loss:
3105.6436 - root_mean_squared_error: 6792.6108
Epoch 98/100
34/34 0s 1ms/step - loss:
3097.8511 - root_mean_squared_error: 6808.0435
Epoch 99/100
34/34 0s 1ms/step - loss:
3119.0466 - root_mean_squared_error: 6755.1343
Epoch 100/100
34/34 0s 1ms/step - loss:

```
3091.2427 - root_mean_squared_error: 6772.1113
```

```
[22]: <keras.src.callbacks.history.History at 0x77f9e3f39150>
```

Yes, the loss did decrease. But not that much.

Let's evaluate.

```
[23]: model2.evaluate(X_test, y_test)
```

```
9/9          1s 55ms/step - loss:
2755.1304 - root_mean_squared_error: 6306.9321
```

```
[23]: [2755.13037109375, 6306.93212890625]
```

Very good!! If we compare it with the previous model it should be much better.

```
[24]: model.evaluate(X_test, y_test)
```

```
9/9          0s 2ms/step - loss:
7491.6353 - root_mean_squared_error: 12483.1523
```

```
[24]: [7491.63525390625, 12483.15234375]
```

We just halved the MAE and RMSE. NOICE!

One fun thing we can do is see how the model actually learned by observing the loss value for each epoch.

But the problem is when we ran the model again for 100 epochs the previous history was overwritten.

Now worries, now we know that the second model will perform better so letting it have another go wont hurt it.

```
[25]: model2 = models.Sequential(
    [
        layers.Input(shape=(X_train.shape[1],)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1),
    ]
)

model2.compile(
    optimizer=optimizers.Adam(),
    loss=losses.MeanAbsoluteError(),
    metrics=[metrics.RootMeanSquaredError()],
)

model2.fit(
```

```
X_train,  
y_train,  
epochs=200  
)
```

```
Epoch 1/200  
34/34          2s 19ms/step -  
loss: 13316.1611 - root_mean_squared_error: 17932.5195  
Epoch 2/200  
34/34          0s 1ms/step - loss:  
13230.1465 - root_mean_squared_error: 17863.4023  
Epoch 3/200  
34/34          0s 1ms/step - loss:  
12964.4346 - root_mean_squared_error: 17653.3086  
Epoch 4/200  
34/34          0s 1ms/step - loss:  
12230.8770 - root_mean_squared_error: 17093.4473  
Epoch 5/200  
34/34          0s 1ms/step - loss:  
10614.3232 - root_mean_squared_error: 15849.3682  
Epoch 6/200  
34/34          0s 1ms/step - loss:  
8414.6758 - root_mean_squared_error: 13953.8809  
Epoch 7/200  
34/34          0s 1ms/step - loss:  
7411.2144 - root_mean_squared_error: 12512.3740  
Epoch 8/200  
34/34          0s 1ms/step - loss:  
7365.7007 - root_mean_squared_error: 12371.2461  
Epoch 9/200  
34/34          0s 1ms/step - loss:  
7338.7490 - root_mean_squared_error: 12359.1523  
Epoch 10/200  
34/34          0s 1ms/step - loss:  
7309.4434 - root_mean_squared_error: 12413.9902  
Epoch 11/200  
34/34          0s 1ms/step - loss:  
7281.6982 - root_mean_squared_error: 12414.1455  
Epoch 12/200  
34/34          0s 1ms/step - loss:  
7252.7012 - root_mean_squared_error: 12365.0869  
Epoch 13/200  
34/34          0s 1ms/step - loss:  
7224.0571 - root_mean_squared_error: 12440.8096  
Epoch 14/200  
34/34          0s 1ms/step - loss:  
7193.6675 - root_mean_squared_error: 12483.5957
```

Epoch 15/200
34/34 0s 1ms/step - loss:
7160.0942 - root_mean_squared_error: 12410.9775
Epoch 16/200
34/34 0s 1ms/step - loss:
7124.5493 - root_mean_squared_error: 12457.0938
Epoch 17/200
34/34 0s 1ms/step - loss:
7094.9111 - root_mean_squared_error: 12494.6582
Epoch 18/200
34/34 0s 1ms/step - loss:
7055.7505 - root_mean_squared_error: 12472.9844
Epoch 19/200
34/34 0s 1ms/step - loss:
7019.1157 - root_mean_squared_error: 12476.8457
Epoch 20/200
34/34 0s 1ms/step - loss:
6982.2236 - root_mean_squared_error: 12544.0908
Epoch 21/200
34/34 0s 1ms/step - loss:
6945.7764 - root_mean_squared_error: 12489.5479
Epoch 22/200
34/34 0s 1ms/step - loss:
6908.6641 - root_mean_squared_error: 12573.8203
Epoch 23/200
34/34 0s 1ms/step - loss:
6867.4517 - root_mean_squared_error: 12614.2168
Epoch 24/200
34/34 0s 1ms/step - loss:
6826.4746 - root_mean_squared_error: 12585.2461
Epoch 25/200
34/34 0s 1ms/step - loss:
6785.3691 - root_mean_squared_error: 12629.8662
Epoch 26/200
34/34 0s 1ms/step - loss:
6751.2080 - root_mean_squared_error: 12677.6260
Epoch 27/200
34/34 0s 1ms/step - loss:
6720.9878 - root_mean_squared_error: 12662.9277
Epoch 28/200
34/34 0s 1ms/step - loss:
6682.0591 - root_mean_squared_error: 12637.0771
Epoch 29/200
34/34 0s 1ms/step - loss:
6662.3657 - root_mean_squared_error: 12720.1260
Epoch 30/200
34/34 0s 1ms/step - loss:
6654.9141 - root_mean_squared_error: 12712.3564

Epoch 31/200
34/34 0s 1ms/step - loss:
6624.8760 - root_mean_squared_error: 12671.4375
Epoch 32/200
34/34 0s 1ms/step - loss:
6611.7310 - root_mean_squared_error: 12692.6055
Epoch 33/200
34/34 0s 1ms/step - loss:
6604.1450 - root_mean_squared_error: 12712.6494
Epoch 34/200
34/34 0s 1ms/step - loss:
6594.9165 - root_mean_squared_error: 12678.8008
Epoch 35/200
34/34 0s 1ms/step - loss:
6582.8657 - root_mean_squared_error: 12698.3604
Epoch 36/200
34/34 0s 1ms/step - loss:
6577.4946 - root_mean_squared_error: 12674.8252
Epoch 37/200
34/34 0s 1ms/step - loss:
6567.4927 - root_mean_squared_error: 12668.3438
Epoch 38/200
34/34 0s 1ms/step - loss:
6554.6914 - root_mean_squared_error: 12637.9580
Epoch 39/200
34/34 0s 1ms/step - loss:
6551.2690 - root_mean_squared_error: 12662.3369
Epoch 40/200
34/34 0s 1ms/step - loss:
6543.8877 - root_mean_squared_error: 12644.6211
Epoch 41/200
34/34 0s 1ms/step - loss:
6534.4214 - root_mean_squared_error: 12619.1367
Epoch 42/200
34/34 0s 1ms/step - loss:
6523.2227 - root_mean_squared_error: 12598.1328
Epoch 43/200
34/34 0s 1ms/step - loss:
6514.9316 - root_mean_squared_error: 12613.3105
Epoch 44/200
34/34 0s 1ms/step - loss:
6504.5430 - root_mean_squared_error: 12592.9463
Epoch 45/200
34/34 0s 1ms/step - loss:
6495.9653 - root_mean_squared_error: 12610.8682
Epoch 46/200
34/34 0s 1ms/step - loss:
6491.4272 - root_mean_squared_error: 12553.9629

Epoch 47/200
34/34 0s 1ms/step - loss:
6478.5977 - root_mean_squared_error: 12573.5234
Epoch 48/200
34/34 0s 1ms/step - loss:
6468.6079 - root_mean_squared_error: 12560.3916
Epoch 49/200
34/34 0s 1ms/step - loss:
6456.1909 - root_mean_squared_error: 12520.7148
Epoch 50/200
34/34 0s 1ms/step - loss:
6447.6665 - root_mean_squared_error: 12545.3135
Epoch 51/200
34/34 0s 1ms/step - loss:
6437.6851 - root_mean_squared_error: 12513.7441
Epoch 52/200
34/34 0s 1ms/step - loss:
6426.9185 - root_mean_squared_error: 12531.7002
Epoch 53/200
34/34 0s 1ms/step - loss:
6419.8262 - root_mean_squared_error: 12491.9287
Epoch 54/200
34/34 0s 1ms/step - loss:
6406.2310 - root_mean_squared_error: 12495.5498
Epoch 55/200
34/34 0s 1ms/step - loss:
6399.0488 - root_mean_squared_error: 12456.6152
Epoch 56/200
34/34 0s 1ms/step - loss:
6385.8003 - root_mean_squared_error: 12460.1611
Epoch 57/200
34/34 0s 1ms/step - loss:
6379.8140 - root_mean_squared_error: 12429.6201
Epoch 58/200
34/34 0s 1ms/step - loss:
6363.8481 - root_mean_squared_error: 12425.6113
Epoch 59/200
34/34 0s 1ms/step - loss:
6358.9707 - root_mean_squared_error: 12397.8506
Epoch 60/200
34/34 0s 1ms/step - loss:
6336.7930 - root_mean_squared_error: 12409.6094
Epoch 61/200
34/34 0s 1ms/step - loss:
6340.0571 - root_mean_squared_error: 12367.6836
Epoch 62/200
34/34 0s 1ms/step - loss:
6313.2251 - root_mean_squared_error: 12340.5088

Epoch 63/200
34/34 0s 1ms/step - loss:
6301.4326 - root_mean_squared_error: 12337.1221
Epoch 64/200
34/34 0s 1ms/step - loss:
6289.7075 - root_mean_squared_error: 12309.1260
Epoch 65/200
34/34 0s 1ms/step - loss:
6271.7012 - root_mean_squared_error: 12291.6445
Epoch 66/200
34/34 0s 2ms/step - loss:
6257.7847 - root_mean_squared_error: 12270.5234
Epoch 67/200
34/34 0s 1ms/step - loss:
6253.4971 - root_mean_squared_error: 12256.7861
Epoch 68/200
34/34 0s 1ms/step - loss:
6230.9751 - root_mean_squared_error: 12215.2832
Epoch 69/200
34/34 0s 1ms/step - loss:
6216.3418 - root_mean_squared_error: 12177.3984
Epoch 70/200
34/34 0s 1ms/step - loss:
6197.1211 - root_mean_squared_error: 12182.3232
Epoch 71/200
34/34 0s 1ms/step - loss:
6180.4458 - root_mean_squared_error: 12152.4287
Epoch 72/200
34/34 0s 1ms/step - loss:
6163.6968 - root_mean_squared_error: 12098.9443
Epoch 73/200
34/34 0s 1ms/step - loss:
6144.4160 - root_mean_squared_error: 12083.7490
Epoch 74/200
34/34 0s 1ms/step - loss:
6121.2920 - root_mean_squared_error: 12033.7832
Epoch 75/200
34/34 0s 1ms/step - loss:
6100.1104 - root_mean_squared_error: 12015.3672
Epoch 76/200
34/34 0s 1ms/step - loss:
6081.8257 - root_mean_squared_error: 11989.3428
Epoch 77/200
34/34 0s 1ms/step - loss:
6060.8481 - root_mean_squared_error: 11963.2031
Epoch 78/200
34/34 0s 1ms/step - loss:
6035.1963 - root_mean_squared_error: 11906.4590

Epoch 79/200
 34/34 0s 1ms/step - loss:
 6006.4702 - root_mean_squared_error: 11859.9561
 Epoch 80/200
 34/34 0s 1ms/step - loss:
 5983.4561 - root_mean_squared_error: 11821.7256
 Epoch 81/200
 34/34 0s 1ms/step - loss:
 5955.6113 - root_mean_squared_error: 11790.5469
 Epoch 82/200
 34/34 0s 1ms/step - loss:
 5926.3940 - root_mean_squared_error: 11729.4775
 Epoch 83/200
 34/34 0s 1ms/step - loss:
 5895.7300 - root_mean_squared_error: 11676.9326
 Epoch 84/200
 34/34 0s 1ms/step - loss:
 5861.0972 - root_mean_squared_error: 11642.5176
 Epoch 85/200
 34/34 0s 1ms/step - loss:
 5823.8188 - root_mean_squared_error: 11575.8613
 Epoch 86/200
 34/34 0s 1ms/step - loss:
 5785.8062 - root_mean_squared_error: 11518.1113
 Epoch 87/200
 34/34 0s 1ms/step - loss:
 5746.2905 - root_mean_squared_error: 11445.3320
 Epoch 88/200
 34/34 0s 1ms/step - loss:
 5697.7983 - root_mean_squared_error: 11377.9004
 Epoch 89/200
 34/34 0s 1ms/step - loss:
 5650.9233 - root_mean_squared_error: 11283.2607
 Epoch 90/200
 34/34 0s 1ms/step - loss:
 5604.4214 - root_mean_squared_error: 11214.8955
 Epoch 91/200
 34/34 0s 1ms/step - loss:
 5548.9307 - root_mean_squared_error: 11102.9326
 Epoch 92/200
 34/34 0s 1ms/step - loss:
 5490.8999 - root_mean_squared_error: 11009.8750
 Epoch 93/200
 34/34 0s 1ms/step - loss:
 5427.6802 - root_mean_squared_error: 10940.6484
 Epoch 94/200
 34/34 0s 1ms/step - loss:
 5359.9395 - root_mean_squared_error: 10795.9189

Epoch 95/200
34/34 0s 2ms/step - loss:
5297.7529 - root_mean_squared_error: 10691.0898
Epoch 96/200
34/34 0s 1ms/step - loss:
5215.1802 - root_mean_squared_error: 10569.4922
Epoch 97/200
34/34 0s 1ms/step - loss:
5117.1841 - root_mean_squared_error: 10402.3525
Epoch 98/200
34/34 0s 1ms/step - loss:
5016.2144 - root_mean_squared_error: 10241.5908
Epoch 99/200
34/34 0s 1ms/step - loss:
4911.0317 - root_mean_squared_error: 10086.3145
Epoch 100/200
34/34 0s 1ms/step - loss:
4802.1528 - root_mean_squared_error: 9878.4072
Epoch 101/200
34/34 0s 1ms/step - loss:
4669.5190 - root_mean_squared_error: 9688.9805
Epoch 102/200
34/34 0s 1ms/step - loss:
4562.3423 - root_mean_squared_error: 9485.4307
Epoch 103/200
34/34 0s 1ms/step - loss:
4414.7524 - root_mean_squared_error: 9272.5469
Epoch 104/200
34/34 0s 1ms/step - loss:
4320.8442 - root_mean_squared_error: 8993.6738
Epoch 105/200
34/34 0s 1ms/step - loss:
4197.1631 - root_mean_squared_error: 8802.8223
Epoch 106/200
34/34 0s 1ms/step - loss:
4088.1841 - root_mean_squared_error: 8589.1055
Epoch 107/200
34/34 0s 1ms/step - loss:
3964.5056 - root_mean_squared_error: 8421.8799
Epoch 108/200
34/34 0s 1ms/step - loss:
3911.3506 - root_mean_squared_error: 8234.5029
Epoch 109/200
34/34 0s 1ms/step - loss:
3835.2485 - root_mean_squared_error: 8085.8721
Epoch 110/200
34/34 0s 1ms/step - loss:
3785.3931 - root_mean_squared_error: 7959.9365

Epoch 111/200
34/34 0s 1ms/step - loss:
3792.2581 - root_mean_squared_error: 7833.9951
Epoch 112/200
34/34 0s 1ms/step - loss:
3709.6533 - root_mean_squared_error: 7762.8516
Epoch 113/200
34/34 0s 1ms/step - loss:
3678.4524 - root_mean_squared_error: 7663.7422
Epoch 114/200
34/34 0s 1ms/step - loss:
3669.7876 - root_mean_squared_error: 7625.9868
Epoch 115/200
34/34 0s 1ms/step - loss:
3650.5835 - root_mean_squared_error: 7577.4077
Epoch 116/200
34/34 0s 1ms/step - loss:
3638.7351 - root_mean_squared_error: 7513.8257
Epoch 117/200
34/34 0s 1ms/step - loss:
3621.4810 - root_mean_squared_error: 7477.8950
Epoch 118/200
34/34 0s 1ms/step - loss:
3618.1995 - root_mean_squared_error: 7443.5293
Epoch 119/200
34/34 0s 1ms/step - loss:
3605.3320 - root_mean_squared_error: 7405.5024
Epoch 120/200
34/34 0s 1ms/step - loss:
3598.9998 - root_mean_squared_error: 7356.5083
Epoch 121/200
34/34 0s 1ms/step - loss:
3594.4170 - root_mean_squared_error: 7340.3550
Epoch 122/200
34/34 0s 1ms/step - loss:
3600.6499 - root_mean_squared_error: 7285.3872
Epoch 123/200
34/34 0s 2ms/step - loss:
3571.3616 - root_mean_squared_error: 7230.0991
Epoch 124/200
34/34 0s 1ms/step - loss:
3567.2537 - root_mean_squared_error: 7199.2686
Epoch 125/200
34/34 0s 1ms/step - loss:
3566.7397 - root_mean_squared_error: 7163.7480
Epoch 126/200
34/34 0s 1ms/step - loss:
3558.0208 - root_mean_squared_error: 7132.7646

Epoch 127/200
34/34 0s 1ms/step - loss:
3549.4087 - root_mean_squared_error: 7098.1353
Epoch 128/200
34/34 0s 1ms/step - loss:
3539.2954 - root_mean_squared_error: 7083.6602
Epoch 129/200
34/34 0s 1ms/step - loss:
3539.8625 - root_mean_squared_error: 7012.5273
Epoch 130/200
34/34 0s 1ms/step - loss:
3522.2109 - root_mean_squared_error: 6978.3101
Epoch 131/200
34/34 0s 1ms/step - loss:
3518.7961 - root_mean_squared_error: 6922.8550
Epoch 132/200
34/34 0s 2ms/step - loss:
3511.2026 - root_mean_squared_error: 6892.4722
Epoch 133/200
34/34 0s 1ms/step - loss:
3518.0876 - root_mean_squared_error: 6871.4321
Epoch 134/200
34/34 0s 1ms/step - loss:
3513.1946 - root_mean_squared_error: 6811.4482
Epoch 135/200
34/34 0s 1ms/step - loss:
3500.9353 - root_mean_squared_error: 6800.4341
Epoch 136/200
34/34 0s 1ms/step - loss:
3493.6582 - root_mean_squared_error: 6765.7085
Epoch 137/200
34/34 0s 1ms/step - loss:
3513.9158 - root_mean_squared_error: 6722.9697
Epoch 138/200
34/34 0s 1ms/step - loss:
3491.0452 - root_mean_squared_error: 6689.4414
Epoch 139/200
34/34 0s 1ms/step - loss:
3476.0420 - root_mean_squared_error: 6665.4185
Epoch 140/200
34/34 0s 1ms/step - loss:
3473.9866 - root_mean_squared_error: 6644.1055
Epoch 141/200
34/34 0s 2ms/step - loss:
3464.1941 - root_mean_squared_error: 6622.6763
Epoch 142/200
34/34 0s 1ms/step - loss:
3462.3623 - root_mean_squared_error: 6588.4321

Epoch 143/200
34/34 0s 1ms/step - loss:
3476.2080 - root_mean_squared_error: 6582.9189
Epoch 144/200
34/34 0s 1ms/step - loss:
3458.1526 - root_mean_squared_error: 6538.2715
Epoch 145/200
34/34 0s 1ms/step - loss:
3443.5432 - root_mean_squared_error: 6536.0508
Epoch 146/200
34/34 0s 1ms/step - loss:
3438.1655 - root_mean_squared_error: 6487.5391
Epoch 147/200
34/34 0s 1ms/step - loss:
3426.1672 - root_mean_squared_error: 6481.7842
Epoch 148/200
34/34 0s 1ms/step - loss:
3427.0637 - root_mean_squared_error: 6452.9893
Epoch 149/200
34/34 0s 1ms/step - loss:
3433.9204 - root_mean_squared_error: 6440.0674
Epoch 150/200
34/34 0s 1ms/step - loss:
3423.7346 - root_mean_squared_error: 6419.5122
Epoch 151/200
34/34 0s 1ms/step - loss:
3425.6230 - root_mean_squared_error: 6410.0269
Epoch 152/200
34/34 0s 1ms/step - loss:
3416.0129 - root_mean_squared_error: 6385.5840
Epoch 153/200
34/34 0s 1ms/step - loss:
3423.6113 - root_mean_squared_error: 6378.4614
Epoch 154/200
34/34 0s 1ms/step - loss:
3418.1538 - root_mean_squared_error: 6374.6343
Epoch 155/200
34/34 0s 1ms/step - loss:
3391.0425 - root_mean_squared_error: 6365.5000
Epoch 156/200
34/34 0s 1ms/step - loss:
3389.2039 - root_mean_squared_error: 6361.5376
Epoch 157/200
34/34 0s 1ms/step - loss:
3381.0437 - root_mean_squared_error: 6355.4658
Epoch 158/200
34/34 0s 1ms/step - loss:
3372.5352 - root_mean_squared_error: 6344.2124

Epoch 159/200
34/34 0s 1ms/step - loss:
3378.6377 - root_mean_squared_error: 6345.6904
Epoch 160/200
34/34 0s 1ms/step - loss:
3403.9070 - root_mean_squared_error: 6327.3394
Epoch 161/200
34/34 0s 1ms/step - loss:
3361.5022 - root_mean_squared_error: 6339.8901
Epoch 162/200
34/34 0s 1ms/step - loss:
3349.3037 - root_mean_squared_error: 6342.1948
Epoch 163/200
34/34 0s 1ms/step - loss:
3354.4673 - root_mean_squared_error: 6338.9189
Epoch 164/200
34/34 0s 1ms/step - loss:
3349.8777 - root_mean_squared_error: 6354.3721
Epoch 165/200
34/34 0s 1ms/step - loss:
3348.7861 - root_mean_squared_error: 6354.0161
Epoch 166/200
34/34 0s 2ms/step - loss:
3338.4707 - root_mean_squared_error: 6357.6973
Epoch 167/200
34/34 0s 1ms/step - loss:
3328.4976 - root_mean_squared_error: 6377.1572
Epoch 168/200
34/34 0s 1ms/step - loss:
3328.9109 - root_mean_squared_error: 6367.4131
Epoch 169/200
34/34 0s 1ms/step - loss:
3324.1233 - root_mean_squared_error: 6386.3730
Epoch 170/200
34/34 0s 1ms/step - loss:
3323.0291 - root_mean_squared_error: 6384.8506
Epoch 171/200
34/34 0s 1ms/step - loss:
3309.8538 - root_mean_squared_error: 6413.9370
Epoch 172/200
34/34 0s 1ms/step - loss:
3305.3997 - root_mean_squared_error: 6422.4766
Epoch 173/200
34/34 0s 1ms/step - loss:
3317.9050 - root_mean_squared_error: 6454.7646
Epoch 174/200
34/34 0s 1ms/step - loss:
3324.1621 - root_mean_squared_error: 6443.3193

Epoch 175/200
34/34 0s 1ms/step - loss:
3344.3452 - root_mean_squared_error: 6459.4404
Epoch 176/200
34/34 0s 1ms/step - loss:
3308.3237 - root_mean_squared_error: 6481.0869
Epoch 177/200
34/34 0s 1ms/step - loss:
3290.7009 - root_mean_squared_error: 6494.6816
Epoch 178/200
34/34 0s 1ms/step - loss:
3291.4626 - root_mean_squared_error: 6526.3926
Epoch 179/200
34/34 0s 1ms/step - loss:
3290.6399 - root_mean_squared_error: 6536.1841
Epoch 180/200
34/34 0s 1ms/step - loss:
3275.2842 - root_mean_squared_error: 6560.7065
Epoch 181/200
34/34 0s 1ms/step - loss:
3286.0879 - root_mean_squared_error: 6592.2471
Epoch 182/200
34/34 0s 1ms/step - loss:
3280.4768 - root_mean_squared_error: 6602.6685
Epoch 183/200
34/34 0s 1ms/step - loss:
3284.0964 - root_mean_squared_error: 6616.0420
Epoch 184/200
34/34 0s 1ms/step - loss:
3282.9597 - root_mean_squared_error: 6643.3042
Epoch 185/200
34/34 0s 1ms/step - loss:
3260.6123 - root_mean_squared_error: 6680.9043
Epoch 186/200
34/34 0s 1ms/step - loss:
3258.4080 - root_mean_squared_error: 6703.5923
Epoch 187/200
34/34 0s 1ms/step - loss:
3250.5400 - root_mean_squared_error: 6752.5835
Epoch 188/200
34/34 0s 1ms/step - loss:
3264.6399 - root_mean_squared_error: 6758.6958
Epoch 189/200
34/34 0s 1ms/step - loss:
3271.1985 - root_mean_squared_error: 6750.8398
Epoch 190/200
34/34 0s 2ms/step - loss:
3244.7029 - root_mean_squared_error: 6787.8662

```

Epoch 191/200
34/34          0s 1ms/step - loss:
3252.4751 - root_mean_squared_error: 6787.2056
Epoch 192/200
34/34          0s 1ms/step - loss:
3252.3613 - root_mean_squared_error: 6810.0840
Epoch 193/200
34/34          0s 1ms/step - loss:
3245.6594 - root_mean_squared_error: 6780.4653
Epoch 194/200
34/34          0s 1ms/step - loss:
3259.2876 - root_mean_squared_error: 6816.6880
Epoch 195/200
34/34          0s 1ms/step - loss:
3251.9661 - root_mean_squared_error: 6789.8711
Epoch 196/200
34/34          0s 1ms/step - loss:
3233.8062 - root_mean_squared_error: 6805.6309
Epoch 197/200
34/34          0s 1ms/step - loss:
3242.0527 - root_mean_squared_error: 6816.9165
Epoch 198/200
34/34          0s 1ms/step - loss:
3229.6470 - root_mean_squared_error: 6815.1558
Epoch 199/200
34/34          0s 1ms/step - loss:
3243.7754 - root_mean_squared_error: 6808.5117
Epoch 200/200
34/34          0s 1ms/step - loss:
3230.5527 - root_mean_squared_error: 6804.7031

```

[25]: <keras.src.callbacks.history.History at 0x77f9dc181990>

Now, we can see how the loss has decreased.

[26]: `model2.history.history`

```

[26]: {'loss': [13316.1611328125,
13230.146484375,
12964.4345703125,
12230.876953125,
10614.3232421875,
8414.67578125,
7411.21435546875,
7365.70068359375,
7338.7490234375,
7309.443359375,
7281.6982421875,

```

7252.701171875,
7224.05712890625,
7193.66748046875,
7160.09423828125,
7124.54931640625,
7094.9111328125,
7055.75048828125,
7019.11572265625,
6982.2236328125,
6945.7763671875,
6908.6640625,
6867.45166015625,
6826.474609375,
6785.369140625,
6751.2080078125,
6720.98779296875,
6682.05908203125,
6662.36572265625,
6654.9140625,
6624.8759765625,
6611.73095703125,
6604.14501953125,
6594.91650390625,
6582.86572265625,
6577.49462890625,
6567.49267578125,
6554.69140625,
6551.26904296875,
6543.8876953125,
6534.42138671875,
6523.22265625,
6514.931640625,
6504.54296875,
6495.96533203125,
6491.42724609375,
6478.59765625,
6468.60791015625,
6456.19091796875,
6447.66650390625,
6437.68505859375,
6426.91845703125,
6419.826171875,
6406.23095703125,
6399.048828125,
6385.80029296875,
6379.81396484375,
6363.84814453125,

6358.970703125,
6336.79296875,
6340.05712890625,
6313.22509765625,
6301.4326171875,
6289.70751953125,
6271.701171875,
6257.78466796875,
6253.4970703125,
6230.97509765625,
6216.341796875,
6197.12109375,
6180.44580078125,
6163.69677734375,
6144.416015625,
6121.2919921875,
6100.1103515625,
6081.82568359375,
6060.84814453125,
6035.1962890625,
6006.47021484375,
5983.4560546875,
5955.611328125,
5926.39404296875,
5895.72998046875,
5861.09716796875,
5823.81884765625,
5785.80615234375,
5746.29052734375,
5697.79833984375,
5650.92333984375,
5604.42138671875,
5548.9306640625,
5490.89990234375,
5427.68017578125,
5359.939453125,
5297.7529296875,
5215.18017578125,
5117.18408203125,
5016.21435546875,
4911.03173828125,
4802.15283203125,
4669.51904296875,
4562.34228515625,
4414.75244140625,
4320.84423828125,
4197.1630859375,

4088.18408203125,
3964.505615234375,
3911.3505859375,
3835.24853515625,
3785.39306640625,
3792.258056640625,
3709.6533203125,
3678.452392578125,
3669.78759765625,
3650.58349609375,
3638.735107421875,
3621.48095703125,
3618.199462890625,
3605.33203125,
3598.999755859375,
3594.4169921875,
3600.64990234375,
3571.361572265625,
3567.253662109375,
3566.73974609375,
3558.020751953125,
3549.40869140625,
3539.29541015625,
3539.862548828125,
3522.2109375,
3518.796142578125,
3511.20263671875,
3518.087646484375,
3513.194580078125,
3500.935302734375,
3493.658203125,
3513.915771484375,
3491.045166015625,
3476.0419921875,
3473.986572265625,
3464.194091796875,
3462.3623046875,
3476.2080078125,
3458.152587890625,
3443.543212890625,
3438.16552734375,
3426.167236328125,
3427.063720703125,
3433.92041015625,
3423.734619140625,
3425.623046875,
3416.012939453125,

3423.611328125,
3418.15380859375,
3391.04248046875,
3389.203857421875,
3381.043701171875,
3372.53515625,
3378.6376953125,
3403.906982421875,
3361.502197265625,
3349.3037109375,
3354.46728515625,
3349.877685546875,
3348.7861328125,
3338.470703125,
3328.49755859375,
3328.910888671875,
3324.123291015625,
3323.029052734375,
3309.853759765625,
3305.399658203125,
3317.905029296875,
3324.162109375,
3344.34521484375,
3308.32373046875,
3290.700927734375,
3291.462646484375,
3290.639892578125,
3275.2841796875,
3286.087890625,
3280.476806640625,
3284.096435546875,
3282.959716796875,
3260.6123046875,
3258.407958984375,
3250.5400390625,
3264.639892578125,
3271.198486328125,
3244.702880859375,
3252.47509765625,
3252.361328125,
3245.659423828125,
3259.28759765625,
3251.966064453125,
3233.80615234375,
3242.052734375,
3229.64697265625,
3243.775390625,

3230.552734375],
'root_mean_squared_error': [17932.51953125,
17863.40234375,
17653.30859375,
17093.447265625,
15849.3681640625,
13953.880859375,
12512.3740234375,
12371.24609375,
12359.15234375,
12413.990234375,
12414.1455078125,
12365.0869140625,
12440.8095703125,
12483.595703125,
12410.9775390625,
12457.09375,
12494.658203125,
12472.984375,
12476.845703125,
12544.0908203125,
12489.5478515625,
12573.8203125,
12614.216796875,
12585.24609375,
12629.8662109375,
12677.6259765625,
12662.927734375,
12637.0771484375,
12720.1259765625,
12712.3564453125,
12671.4375,
12692.60546875,
12712.6494140625,
12678.80078125,
12698.3603515625,
12674.8251953125,
12668.34375,
12637.9580078125,
12662.3369140625,
12644.62109375,
12619.13671875,
12598.1328125,
12613.310546875,
12592.9462890625,
12610.8681640625,
12553.962890625,

12573.5234375,
12560.3916015625,
12520.71484375,
12545.3134765625,
12513.744140625,
12531.7001953125,
12491.9287109375,
12495.5498046875,
12456.615234375,
12460.1611328125,
12429.6201171875,
12425.611328125,
12397.8505859375,
12409.609375,
12367.68359375,
12340.5087890625,
12337.1220703125,
12309.1259765625,
12291.64453125,
12270.5234375,
12256.7861328125,
12215.283203125,
12177.3984375,
12182.3232421875,
12152.4287109375,
12098.9443359375,
12083.7490234375,
12033.783203125,
12015.3671875,
11989.3427734375,
11963.203125,
11906.458984375,
11859.9560546875,
11821.7255859375,
11790.546875,
11729.4775390625,
11676.9326171875,
11642.517578125,
11575.861328125,
11518.111328125,
11445.33203125,
11377.900390625,
11283.2607421875,
11214.8955078125,
11102.9326171875,
11009.875,
10940.6484375,

10795.9189453125,
10691.08984375,
10569.4921875,
10402.3525390625,
10241.5908203125,
10086.314453125,
9878.4072265625,
9688.98046875,
9485.4306640625,
9272.546875,
8993.673828125,
8802.822265625,
8589.10546875,
8421.8798828125,
8234.5029296875,
8085.8720703125,
7959.9365234375,
7833.9951171875,
7762.8515625,
7663.7421875,
7625.98681640625,
7577.40771484375,
7513.82568359375,
7477.89501953125,
7443.529296875,
7405.50244140625,
7356.50830078125,
7340.35498046875,
7285.38720703125,
7230.09912109375,
7199.2685546875,
7163.748046875,
7132.7646484375,
7098.13525390625,
7083.66015625,
7012.52734375,
6978.31005859375,
6922.85498046875,
6892.47216796875,
6871.43212890625,
6811.4482421875,
6800.43408203125,
6765.70849609375,
6722.9697265625,
6689.44140625,
6665.41845703125,
6644.10546875,

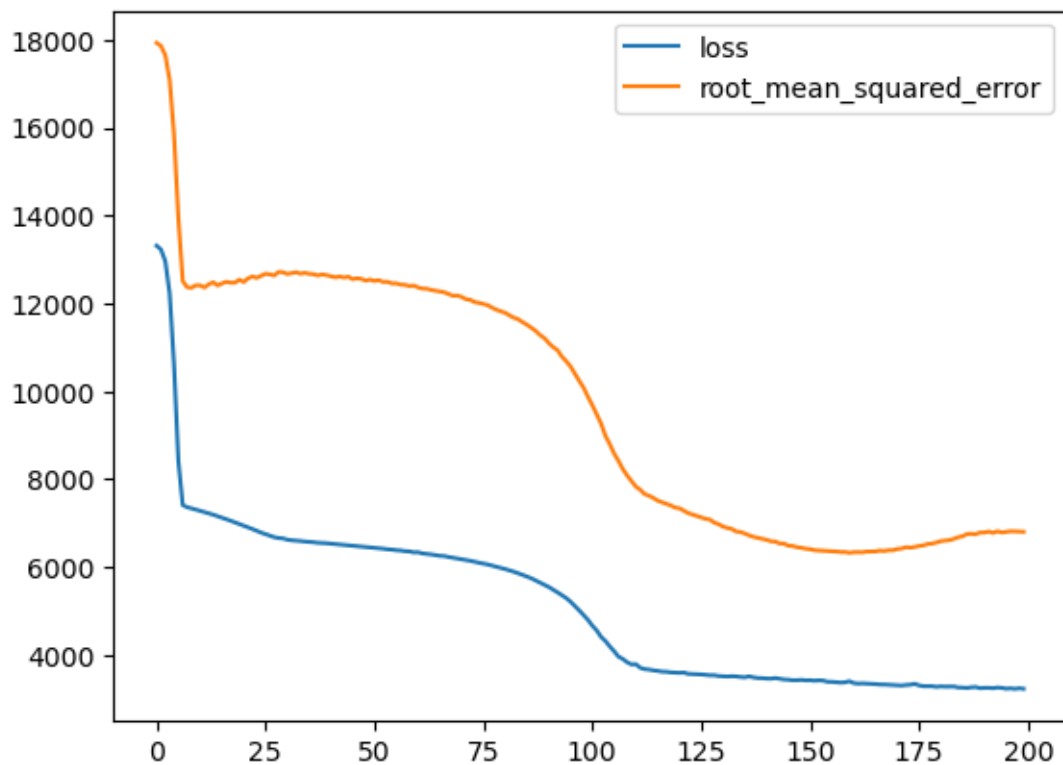
6622.67626953125,
6588.43212890625,
6582.9189453125,
6538.271484375,
6536.05078125,
6487.5390625,
6481.7841796875,
6452.9892578125,
6440.0673828125,
6419.51220703125,
6410.02685546875,
6385.583984375,
6378.46142578125,
6374.63427734375,
6365.5,
6361.53759765625,
6355.4658203125,
6344.21240234375,
6345.6904296875,
6327.33935546875,
6339.89013671875,
6342.19482421875,
6338.9189453125,
6354.3720703125,
6354.01611328125,
6357.697265625,
6377.1572265625,
6367.4130859375,
6386.373046875,
6384.8505859375,
6413.93701171875,
6422.4765625,
6454.7646484375,
6443.3193359375,
6459.4404296875,
6481.0869140625,
6494.681640625,
6526.392578125,
6536.18408203125,
6560.70654296875,
6592.2470703125,
6602.66845703125,
6616.0419921875,
6643.30419921875,
6680.904296875,
6703.59228515625,
6752.58349609375,

```
6758.69580078125,  
6750.83984375,  
6787.8662109375,  
6787.20556640625,  
6810.083984375,  
6780.46533203125,  
6816.68798828125,  
6789.87109375,  
6805.630859375,  
6816.91650390625,  
6815.15576171875,  
6808.51171875,  
6804.703125]}}
```

Seeing the number won't help us much. Let's plot it.

```
[27]: pd.DataFrame(model2.history.history).plot()
```

```
[27]: <Axes: >
```



And if you take a look at the chart the loss(mae) and the RMSE is decreasing but the gap between the two is pretty wide.

Remember the RMSE value fluctuations.

This means the model is learning the training set but it is also giving wrong answers for outliers.

Outliers are data points that are far away from the rest of the data points.

Our model should be able to handle outliers but in this case it is not.

To fix this what we can do is:

- Choose a different loss function(more robust)
- Remove the outliers.

Removing the outliers is not easy and can be very tedious. And we will go into talking about more robust loss functions later.

One thing I want to talk about before ending this is scaling the data.

1.5 Scaling

It is proven that a neural network can be very sensitive to data with different scales. Like a data set with values in meters, kgs, years. This can cause the network to get stuck and produce bad results.

It is a best approach to set all the feature data to a common scale.

The default **scaling** method for neural networks is **minmax** scaling.

What does it do? It scales the data to a range between 0 and 1.

Formula is very simple:

$$x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Another common scaling method is **standardization** scaling.

This method sets the mean to 0 and the standard deviation to 1 of a feature.

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

For now, let's use **minmax** scaling and see if the model performs better.

We can use **sklearn** **MinMaxScaler** class.

```
[31]: from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
scaler.set_output(transform="pandas")
```

```
scaler.fit_transform(X)
```

```
[31]:
```

	age	bmi	children	sex_male	smoker_yes	region_northwest	\
0	0.021739	0.321227	0.0	0.0	1.0	0.0	

1	0.000000	0.479150	0.2	1.0	0.0	0.0
2	0.217391	0.458434	0.6	1.0	0.0	0.0
3	0.326087	0.181464	0.0	1.0	0.0	1.0
4	0.304348	0.347592	0.0	1.0	0.0	1.0
...	...	...	...	...	...	...
1333	0.695652	0.403820	0.6	1.0	0.0	1.0
1334	0.000000	0.429379	0.0	0.0	0.0	0.0
1335	0.000000	0.562012	0.0	0.0	0.0	0.0
1336	0.065217	0.264730	0.0	0.0	0.0	0.0
1337	0.934783	0.352704	0.0	0.0	1.0	1.0

	region_southeast	region_southwest
0	0.0	1.0
1	1.0	0.0
2	1.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
1333	0.0	0.0
1334	0.0	0.0
1335	1.0	0.0
1336	0.0	1.0
1337	0.0	0.0

[1338 rows x 8 columns]

Works nicely right?

But the problem is we cannot tell the model to scale only `age`, `bmi` and `children` columns.

It'll scale all the columns. Even though it's not a big issue but it is still not the best practice.

Best practice would be not set up a `column transformer` for scaling and for `one-hot encoding` at the same time.

We can do that with another class from `sklearn` called `ColumnTransformer` class.

I want to show you the whole thing so let's load the data again.

```
[32]: import pandas as pd

data = pd.read_csv(
    "https://raw.githubusercontent.com/RishatTalukder/
↳Deep_Learning_With_Tensorflow/refs/heads/main/2_regression/insurance.csv"
)
data.head()
```

```
[32]:   age    sex    bmi  children  smoker    region    charges
0   19  female  27.900         0     yes  southwest  16884.92400
1   18   male  33.770         1     no   southeast  1725.55230
```

2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Now we can use ColumnTransformer class.

```
[36]: from sklearn.compose import ColumnTransformer

ct = ColumnTransformer(
    [
        ("scaler", MinMaxScaler(), ["age", "bmi", "children"]),
        ("onehot", OneHotEncoder(drop="first", sparse_output=False), ["sex", "smoker", "region"]),
    ],
)

```

We now use the ct object to transform the data.

We can transform the whole dataset at once. But first we have to separate the features and the labels.

```
[37]: X = data.drop(
        columns=["charges"],
    )

y = data["charges"]

```

And now we fit_transform.

```
[39]: ct.set_output(transform="pandas")
scaled_X = ct.fit_transform(X)
scaled_X

```

```
[39]:
```

	scaler__age	scaler__bmi	scaler__children	onehot__sex_male	\
0	0.021739	0.321227	0.0	0.0	
1	0.000000	0.479150	0.2	1.0	
2	0.217391	0.458434	0.6	1.0	
3	0.326087	0.181464	0.0	1.0	
4	0.304348	0.347592	0.0	1.0	
...	...	...	...	...	
1333	0.695652	0.403820	0.6	1.0	
1334	0.000000	0.429379	0.0	0.0	
1335	0.000000	0.562012	0.0	0.0	
1336	0.065217	0.264730	0.0	0.0	
1337	0.934783	0.352704	0.0	0.0	

	onehot__smoker_yes	onehot__region_northwest	onehot__region_southeast	\
0	1.0	0.0	0.0	

1	0.0	0.0	1.0
2	0.0	0.0	1.0
3	0.0	1.0	0.0
4	0.0	1.0	0.0
...	...	...	...
1333	0.0	1.0	0.0
1334	0.0	0.0	0.0
1335	0.0	0.0	1.0
1336	0.0	0.0	0.0
1337	1.0	1.0	0.0

	onehot__region_southwest
0	1.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
1333	0.0
1334	0.0
1335	0.0
1336	1.0
1337	0.0

[1338 rows x 8 columns]

And we are done.

we can now split the normalized data and train the model and see if the model preforms any better or not.

```
[40]: X_train, X_test, y_train, y_test = train_test_split(
        scaled_X,
        y,
        test_size=0.2,
        random_state=42
    )
```

Now, we can train the model.

I'll make some changes to the model. I'll increase the number of neuron count to 64, 32, 32, 1 for the hidden layers.

```
[42]: model3 = models.Sequential(
        [
            layers.Input(shape=(X_train.shape[1],)),
            layers.Dense(64, activation="relu"),
            layers.Dense(32, activation="relu"),
            layers.Dense(32, activation="relu"),
```



```

        layers.Dense(1),
    ]
)

model3.compile(
    optimizer=optimizers.Adam(),
    loss=losses.MeanAbsoluteError(),
    metrics=[metrics.RootMeanSquaredError()],
)

model3.fit(
    X_train,
    y_train,
    epochs=200
)

```

```

Epoch 1/200
34/34          2s 23ms/step -
loss: 13345.4990 - root_mean_squared_error: 17956.4922
Epoch 2/200
34/34          0s 1ms/step - loss:
13340.2324 - root_mean_squared_error: 17952.1211
Epoch 3/200
34/34          0s 1ms/step - loss:
13306.7686 - root_mean_squared_error: 17924.0996
Epoch 4/200
34/34          0s 1ms/step - loss:
13166.2090 - root_mean_squared_error: 17806.1328
Epoch 5/200
34/34          0s 1ms/step - loss:
12754.9551 - root_mean_squared_error: 17463.8770
Epoch 6/200
34/34          0s 1ms/step - loss:
11836.9004 - root_mean_squared_error: 16704.6543
Epoch 7/200
34/34          0s 1ms/step - loss:
10315.7275 - root_mean_squared_error: 15396.2969
Epoch 8/200
34/34          0s 1ms/step - loss:
8662.1260 - root_mean_squared_error: 13664.5830
Epoch 9/200
34/34          0s 1ms/step - loss:
7678.9849 - root_mean_squared_error: 12191.7871
Epoch 10/200
34/34          0s 1ms/step - loss:
7379.6685 - root_mean_squared_error: 11395.3828
Epoch 11/200

```

```

34/34          0s 1ms/step - loss:
7236.2275 - root_mean_squared_error: 11133.2715
Epoch 12/200
34/34          0s 1ms/step - loss:
7117.8989 - root_mean_squared_error: 10880.9854
Epoch 13/200
34/34          0s 1ms/step - loss:
6990.9434 - root_mean_squared_error: 10729.2236
Epoch 14/200
34/34          0s 1ms/step - loss:
6865.5635 - root_mean_squared_error: 10612.5566
Epoch 15/200
34/34          0s 1ms/step - loss:
6727.9790 - root_mean_squared_error: 10375.9268
Epoch 16/200
34/34          0s 1ms/step - loss:
6585.4409 - root_mean_squared_error: 10195.8740
Epoch 17/200
34/34          0s 1ms/step - loss:
6425.4541 - root_mean_squared_error: 9989.1553
Epoch 18/200
34/34          0s 1ms/step - loss:
6250.0928 - root_mean_squared_error: 9818.3154
Epoch 19/200
34/34          0s 1ms/step - loss:
6055.5068 - root_mean_squared_error: 9608.3359
Epoch 20/200
34/34          0s 1ms/step - loss:
5838.6431 - root_mean_squared_error: 9359.8389
Epoch 21/200
34/34          0s 1ms/step - loss:
5593.6147 - root_mean_squared_error: 9131.8740
Epoch 22/200
34/34          0s 1ms/step - loss:
5320.4248 - root_mean_squared_error: 8818.6768
Epoch 23/200
34/34          0s 1ms/step - loss:
5023.3892 - root_mean_squared_error: 8521.9551
Epoch 24/200
34/34          0s 1ms/step - loss:
4720.8677 - root_mean_squared_error: 8289.9277
Epoch 25/200
34/34          0s 1ms/step - loss:
4429.3706 - root_mean_squared_error: 8031.6416
Epoch 26/200
34/34          0s 2ms/step - loss:
4177.5415 - root_mean_squared_error: 7880.5527
Epoch 27/200

```

34/34 0s 1ms/step - loss:
 3965.9683 - root_mean_squared_error: 7687.9521
 Epoch 28/200
 34/34 0s 1ms/step - loss:
 3796.2710 - root_mean_squared_error: 7503.6421
 Epoch 29/200
 34/34 0s 1ms/step - loss:
 3697.7019 - root_mean_squared_error: 7390.8184
 Epoch 30/200
 34/34 0s 1ms/step - loss:
 3646.6143 - root_mean_squared_error: 7304.5840
 Epoch 31/200
 34/34 0s 1ms/step - loss:
 3624.5684 - root_mean_squared_error: 7233.1021
 Epoch 32/200
 34/34 0s 1ms/step - loss:
 3609.8940 - root_mean_squared_error: 7168.8911
 Epoch 33/200
 34/34 0s 1ms/step - loss:
 3594.9343 - root_mean_squared_error: 7119.3818
 Epoch 34/200
 34/34 0s 1ms/step - loss:
 3584.2063 - root_mean_squared_error: 7042.0020
 Epoch 35/200
 34/34 0s 1ms/step - loss:
 3567.4502 - root_mean_squared_error: 6974.9180
 Epoch 36/200
 34/34 0s 1ms/step - loss:
 3553.5676 - root_mean_squared_error: 6923.0762
 Epoch 37/200
 34/34 0s 1ms/step - loss:
 3540.9465 - root_mean_squared_error: 6869.2676
 Epoch 38/200
 34/34 0s 1ms/step - loss:
 3533.7407 - root_mean_squared_error: 6827.2939
 Epoch 39/200
 34/34 0s 1ms/step - loss:
 3519.3271 - root_mean_squared_error: 6782.8975
 Epoch 40/200
 34/34 0s 1ms/step - loss:
 3507.9568 - root_mean_squared_error: 6748.1348
 Epoch 41/200
 34/34 0s 1ms/step - loss:
 3494.4316 - root_mean_squared_error: 6695.6689
 Epoch 42/200
 34/34 0s 1ms/step - loss:
 3484.3447 - root_mean_squared_error: 6662.7905
 Epoch 43/200

34/34 0s 1ms/step - loss:
3473.9731 - root_mean_squared_error: 6641.3379
Epoch 44/200
34/34 0s 1ms/step - loss:
3460.5886 - root_mean_squared_error: 6603.3022
Epoch 45/200
34/34 0s 1ms/step - loss:
3450.0122 - root_mean_squared_error: 6574.7837
Epoch 46/200
34/34 0s 1ms/step - loss:
3436.7988 - root_mean_squared_error: 6552.2446
Epoch 47/200
34/34 0s 1ms/step - loss:
3422.3049 - root_mean_squared_error: 6540.4673
Epoch 48/200
34/34 0s 1ms/step - loss:
3413.2590 - root_mean_squared_error: 6536.1274
Epoch 49/200
34/34 0s 1ms/step - loss:
3399.6025 - root_mean_squared_error: 6512.6558
Epoch 50/200
34/34 0s 1ms/step - loss:
3389.8279 - root_mean_squared_error: 6515.7388
Epoch 51/200
34/34 0s 1ms/step - loss:
3379.1694 - root_mean_squared_error: 6530.3232
Epoch 52/200
34/34 0s 1ms/step - loss:
3368.3030 - root_mean_squared_error: 6538.9863
Epoch 53/200
34/34 0s 1ms/step - loss:
3357.2300 - root_mean_squared_error: 6550.9185
Epoch 54/200
34/34 0s 1ms/step - loss:
3355.6594 - root_mean_squared_error: 6589.7720
Epoch 55/200
34/34 0s 1ms/step - loss:
3339.9739 - root_mean_squared_error: 6594.5166
Epoch 56/200
34/34 0s 1ms/step - loss:
3336.7976 - root_mean_squared_error: 6595.1641
Epoch 57/200
34/34 0s 1ms/step - loss:
3319.7549 - root_mean_squared_error: 6633.6680
Epoch 58/200
34/34 0s 1ms/step - loss:
3311.4763 - root_mean_squared_error: 6676.0767
Epoch 59/200

```

34/34          0s 1ms/step - loss:
3301.1379 - root_mean_squared_error: 6727.6074
Epoch 60/200
34/34          0s 1ms/step - loss:
3294.0618 - root_mean_squared_error: 6772.6968
Epoch 61/200
34/34          0s 1ms/step - loss:
3287.8567 - root_mean_squared_error: 6823.2388
Epoch 62/200
34/34          0s 1ms/step - loss:
3282.1235 - root_mean_squared_error: 6859.8828
Epoch 63/200
34/34          0s 1ms/step - loss:
3278.6089 - root_mean_squared_error: 6931.7310
Epoch 64/200
34/34          0s 1ms/step - loss:
3264.6323 - root_mean_squared_error: 6936.7231
Epoch 65/200
34/34          0s 1ms/step - loss:
3264.4458 - root_mean_squared_error: 6969.7827
Epoch 66/200
34/34          0s 1ms/step - loss:
3259.3020 - root_mean_squared_error: 6968.8188
Epoch 67/200
34/34          0s 1ms/step - loss:
3260.4871 - root_mean_squared_error: 6993.6318
Epoch 68/200
34/34          0s 1ms/step - loss:
3252.9438 - root_mean_squared_error: 6995.6924
Epoch 69/200
34/34          0s 1ms/step - loss:
3252.8936 - root_mean_squared_error: 6989.4150
Epoch 70/200
34/34          0s 1ms/step - loss:
3249.7866 - root_mean_squared_error: 6990.8154
Epoch 71/200
34/34          0s 1ms/step - loss:
3245.8835 - root_mean_squared_error: 7013.6025
Epoch 72/200
34/34          0s 1ms/step - loss:
3246.4761 - root_mean_squared_error: 7005.0894
Epoch 73/200
34/34          0s 1ms/step - loss:
3243.8071 - root_mean_squared_error: 7006.3267
Epoch 74/200
34/34          0s 1ms/step - loss:
3238.5906 - root_mean_squared_error: 6998.8770
Epoch 75/200

```

34/34 0s 1ms/step - loss:
 3238.7195 - root_mean_squared_error: 7014.9458
 Epoch 76/200
 34/34 0s 1ms/step - loss:
 3237.5674 - root_mean_squared_error: 7030.9365
 Epoch 77/200
 34/34 0s 1ms/step - loss:
 3232.8333 - root_mean_squared_error: 7017.3325
 Epoch 78/200
 34/34 0s 1ms/step - loss:
 3236.2246 - root_mean_squared_error: 6980.4692
 Epoch 79/200
 34/34 0s 1ms/step - loss:
 3233.4897 - root_mean_squared_error: 6986.3179
 Epoch 80/200
 34/34 0s 1ms/step - loss:
 3228.5955 - root_mean_squared_error: 6993.8638
 Epoch 81/200
 34/34 0s 1ms/step - loss:
 3231.9402 - root_mean_squared_error: 7052.6357
 Epoch 82/200
 34/34 0s 1ms/step - loss:
 3229.2534 - root_mean_squared_error: 7020.5020
 Epoch 83/200
 34/34 0s 1ms/step - loss:
 3226.9055 - root_mean_squared_error: 6996.7905
 Epoch 84/200
 34/34 0s 1ms/step - loss:
 3234.1870 - root_mean_squared_error: 6988.0244
 Epoch 85/200
 34/34 0s 1ms/step - loss:
 3227.0779 - root_mean_squared_error: 6999.7617
 Epoch 86/200
 34/34 0s 1ms/step - loss:
 3222.0745 - root_mean_squared_error: 7009.2896
 Epoch 87/200
 34/34 0s 1ms/step - loss:
 3218.9475 - root_mean_squared_error: 6994.9976
 Epoch 88/200
 34/34 0s 1ms/step - loss:
 3222.3911 - root_mean_squared_error: 6998.9805
 Epoch 89/200
 34/34 0s 1ms/step - loss:
 3217.4839 - root_mean_squared_error: 7006.9722
 Epoch 90/200
 34/34 0s 1ms/step - loss:
 3222.6963 - root_mean_squared_error: 6993.5010
 Epoch 91/200

```

34/34          0s 1ms/step - loss:
3215.6631 - root_mean_squared_error: 6991.7290
Epoch 92/200
34/34          0s 1ms/step - loss:
3213.0959 - root_mean_squared_error: 6997.1851
Epoch 93/200
34/34          0s 1ms/step - loss:
3215.7891 - root_mean_squared_error: 6964.5420
Epoch 94/200
34/34          0s 1ms/step - loss:
3224.1365 - root_mean_squared_error: 6999.1191
Epoch 95/200
34/34          0s 1ms/step - loss:
3216.8640 - root_mean_squared_error: 6985.5225
Epoch 96/200
34/34          0s 1ms/step - loss:
3213.4497 - root_mean_squared_error: 6962.0347
Epoch 97/200
34/34          0s 1ms/step - loss:
3215.2246 - root_mean_squared_error: 6951.6973
Epoch 98/200
34/34          0s 1ms/step - loss:
3212.3372 - root_mean_squared_error: 6974.1567
Epoch 99/200
34/34          0s 1ms/step - loss:
3214.5713 - root_mean_squared_error: 6988.0430
Epoch 100/200
34/34          0s 1ms/step - loss:
3204.3577 - root_mean_squared_error: 6971.1841
Epoch 101/200
34/34          0s 1ms/step - loss:
3205.3503 - root_mean_squared_error: 6990.3394
Epoch 102/200
34/34          0s 1ms/step - loss:
3203.0298 - root_mean_squared_error: 6962.7930
Epoch 103/200
34/34          0s 1ms/step - loss:
3202.4695 - root_mean_squared_error: 6976.9692
Epoch 104/200
34/34          0s 1ms/step - loss:
3202.4133 - root_mean_squared_error: 6996.4692
Epoch 105/200
34/34          0s 1ms/step - loss:
3203.8105 - root_mean_squared_error: 6969.9390
Epoch 106/200
34/34          0s 1ms/step - loss:
3201.5010 - root_mean_squared_error: 6967.4180
Epoch 107/200

```

```

34/34          0s 1ms/step - loss:
3203.9109 - root_mean_squared_error: 6959.2339
Epoch 108/200
34/34          0s 1ms/step - loss:
3198.3784 - root_mean_squared_error: 6957.6997
Epoch 109/200
34/34          0s 1ms/step - loss:
3198.4497 - root_mean_squared_error: 6934.1787
Epoch 110/200
34/34          0s 1ms/step - loss:
3196.2422 - root_mean_squared_error: 6956.9995
Epoch 111/200
34/34          0s 1ms/step - loss:
3199.0737 - root_mean_squared_error: 6977.6401
Epoch 112/200
34/34          0s 1ms/step - loss:
3192.0444 - root_mean_squared_error: 6976.1885
Epoch 113/200
34/34          0s 1ms/step - loss:
3195.6289 - root_mean_squared_error: 6946.9644
Epoch 114/200
34/34          0s 1ms/step - loss:
3191.7061 - root_mean_squared_error: 6980.4692
Epoch 115/200
34/34          0s 1ms/step - loss:
3189.0996 - root_mean_squared_error: 6974.8516
Epoch 116/200
34/34          0s 1ms/step - loss:
3188.5916 - root_mean_squared_error: 6950.3833
Epoch 117/200
34/34          0s 1ms/step - loss:
3187.9805 - root_mean_squared_error: 6970.6211
Epoch 118/200
34/34          0s 1ms/step - loss:
3186.8176 - root_mean_squared_error: 6950.3813
Epoch 119/200
34/34          0s 1ms/step - loss:
3194.0930 - root_mean_squared_error: 6979.1963
Epoch 120/200
34/34          0s 1ms/step - loss:
3193.6431 - root_mean_squared_error: 6964.7368
Epoch 121/200
34/34          0s 1ms/step - loss:
3184.6643 - root_mean_squared_error: 6960.7993
Epoch 122/200
34/34          0s 1ms/step - loss:
3181.8855 - root_mean_squared_error: 6950.2251
Epoch 123/200

```



```

34/34          0s 1ms/step - loss:
3182.5042 - root_mean_squared_error: 6971.3682
Epoch 124/200
34/34          0s 1ms/step - loss:
3183.7605 - root_mean_squared_error: 6928.9580
Epoch 125/200
34/34          0s 1ms/step - loss:
3182.7200 - root_mean_squared_error: 6951.6777
Epoch 126/200
34/34          0s 1ms/step - loss:
3183.8230 - root_mean_squared_error: 6957.5620
Epoch 127/200
34/34          0s 1ms/step - loss:
3178.1189 - root_mean_squared_error: 6955.1836
Epoch 128/200
34/34          0s 1ms/step - loss:
3180.3872 - root_mean_squared_error: 6935.6060
Epoch 129/200
34/34          0s 1ms/step - loss:
3179.5244 - root_mean_squared_error: 6944.4849
Epoch 130/200
34/34          0s 1ms/step - loss:
3178.6050 - root_mean_squared_error: 6937.9580
Epoch 131/200
34/34          0s 1ms/step - loss:
3173.9785 - root_mean_squared_error: 6930.4482
Epoch 132/200
34/34          0s 1ms/step - loss:
3178.3516 - root_mean_squared_error: 6952.7212
Epoch 133/200
34/34          0s 1ms/step - loss:
3173.4351 - root_mean_squared_error: 6921.7500
Epoch 134/200
34/34          0s 1ms/step - loss:
3177.5950 - root_mean_squared_error: 6915.1426
Epoch 135/200
34/34          0s 1ms/step - loss:
3172.6279 - root_mean_squared_error: 6932.3618
Epoch 136/200
34/34          0s 1ms/step - loss:
3177.2959 - root_mean_squared_error: 6935.0366
Epoch 137/200
34/34          0s 1ms/step - loss:
3168.9631 - root_mean_squared_error: 6928.1626
Epoch 138/200
34/34          0s 1ms/step - loss:
3169.6965 - root_mean_squared_error: 6928.5117
Epoch 139/200

```

34/34 0s 1ms/step - loss:
 3169.8721 - root_mean_squared_error: 6929.1831
 Epoch 140/200
 34/34 0s 1ms/step - loss:
 3168.3472 - root_mean_squared_error: 6909.6221
 Epoch 141/200
 34/34 0s 1ms/step - loss:
 3165.6917 - root_mean_squared_error: 6913.5137
 Epoch 142/200
 34/34 0s 1ms/step - loss:
 3165.9941 - root_mean_squared_error: 6928.1494
 Epoch 143/200
 34/34 0s 1ms/step - loss:
 3163.6243 - root_mean_squared_error: 6914.6655
 Epoch 144/200
 34/34 0s 1ms/step - loss:
 3166.1580 - root_mean_squared_error: 6896.1377
 Epoch 145/200
 34/34 0s 2ms/step - loss:
 3169.8289 - root_mean_squared_error: 6927.7300
 Epoch 146/200
 34/34 0s 1ms/step - loss:
 3164.2690 - root_mean_squared_error: 6882.0029
 Epoch 147/200
 34/34 0s 1ms/step - loss:
 3163.6387 - root_mean_squared_error: 6899.8345
 Epoch 148/200
 34/34 0s 1ms/step - loss:
 3160.4814 - root_mean_squared_error: 6925.3096
 Epoch 149/200
 34/34 0s 1ms/step - loss:
 3167.6587 - root_mean_squared_error: 6908.5376
 Epoch 150/200
 34/34 0s 1ms/step - loss:
 3165.2976 - root_mean_squared_error: 6896.7261
 Epoch 151/200
 34/34 0s 1ms/step - loss:
 3159.6990 - root_mean_squared_error: 6913.9824
 Epoch 152/200
 34/34 0s 1ms/step - loss:
 3158.3667 - root_mean_squared_error: 6898.6104
 Epoch 153/200
 34/34 0s 1ms/step - loss:
 3158.1470 - root_mean_squared_error: 6893.1465
 Epoch 154/200
 34/34 0s 1ms/step - loss:
 3153.5127 - root_mean_squared_error: 6896.7168
 Epoch 155/200

```

34/34          0s 1ms/step - loss:
3153.5928 - root_mean_squared_error: 6920.9683
Epoch 156/200
34/34          0s 1ms/step - loss:
3151.9604 - root_mean_squared_error: 6918.2334
Epoch 157/200
34/34          0s 1ms/step - loss:
3151.8635 - root_mean_squared_error: 6860.5464
Epoch 158/200
34/34          0s 1ms/step - loss:
3152.8218 - root_mean_squared_error: 6866.3872
Epoch 159/200
34/34          0s 1ms/step - loss:
3149.6990 - root_mean_squared_error: 6909.5029
Epoch 160/200
34/34          0s 1ms/step - loss:
3149.5691 - root_mean_squared_error: 6880.2363
Epoch 161/200
34/34          0s 1ms/step - loss:
3149.5354 - root_mean_squared_error: 6869.6899
Epoch 162/200
34/34          0s 1ms/step - loss:
3144.7898 - root_mean_squared_error: 6891.8936
Epoch 163/200
34/34          0s 1ms/step - loss:
3145.8450 - root_mean_squared_error: 6869.9082
Epoch 164/200
34/34          0s 1ms/step - loss:
3148.2532 - root_mean_squared_error: 6880.3965
Epoch 165/200
34/34          0s 1ms/step - loss:
3141.1584 - root_mean_squared_error: 6850.6826
Epoch 166/200
34/34          0s 1ms/step - loss:
3139.6868 - root_mean_squared_error: 6870.4980
Epoch 167/200
34/34          0s 1ms/step - loss:
3146.0625 - root_mean_squared_error: 6897.9028
Epoch 168/200
34/34          0s 1ms/step - loss:
3139.2126 - root_mean_squared_error: 6852.8257
Epoch 169/200
34/34          0s 1ms/step - loss:
3142.0520 - root_mean_squared_error: 6855.4888
Epoch 170/200
34/34          0s 1ms/step - loss:
3137.8718 - root_mean_squared_error: 6862.8765
Epoch 171/200

```

34/34 0s 1ms/step - loss:
 3136.5684 - root_mean_squared_error: 6864.0991
 Epoch 172/200
 34/34 0s 1ms/step - loss:
 3135.3284 - root_mean_squared_error: 6855.2456
 Epoch 173/200
 34/34 0s 1ms/step - loss:
 3132.9436 - root_mean_squared_error: 6874.5483
 Epoch 174/200
 34/34 0s 1ms/step - loss:
 3134.9590 - root_mean_squared_error: 6840.0469
 Epoch 175/200
 34/34 0s 1ms/step - loss:
 3133.2490 - root_mean_squared_error: 6857.0200
 Epoch 176/200
 34/34 0s 1ms/step - loss:
 3135.2744 - root_mean_squared_error: 6856.0767
 Epoch 177/200
 34/34 0s 1ms/step - loss:
 3132.3015 - root_mean_squared_error: 6845.7295
 Epoch 178/200
 34/34 0s 1ms/step - loss:
 3130.3860 - root_mean_squared_error: 6859.2690
 Epoch 179/200
 34/34 0s 1ms/step - loss:
 3124.6521 - root_mean_squared_error: 6855.0078
 Epoch 180/200
 34/34 0s 1ms/step - loss:
 3132.4663 - root_mean_squared_error: 6824.4067
 Epoch 181/200
 34/34 0s 1ms/step - loss:
 3124.5359 - root_mean_squared_error: 6844.5840
 Epoch 182/200
 34/34 0s 1ms/step - loss:
 3127.3005 - root_mean_squared_error: 6837.7344
 Epoch 183/200
 34/34 0s 1ms/step - loss:
 3123.2229 - root_mean_squared_error: 6814.1655
 Epoch 184/200
 34/34 0s 1ms/step - loss:
 3120.3679 - root_mean_squared_error: 6842.3291
 Epoch 185/200
 34/34 0s 1ms/step - loss:
 3120.8950 - root_mean_squared_error: 6839.5015
 Epoch 186/200
 34/34 0s 1ms/step - loss:
 3122.3760 - root_mean_squared_error: 6800.5308
 Epoch 187/200

```

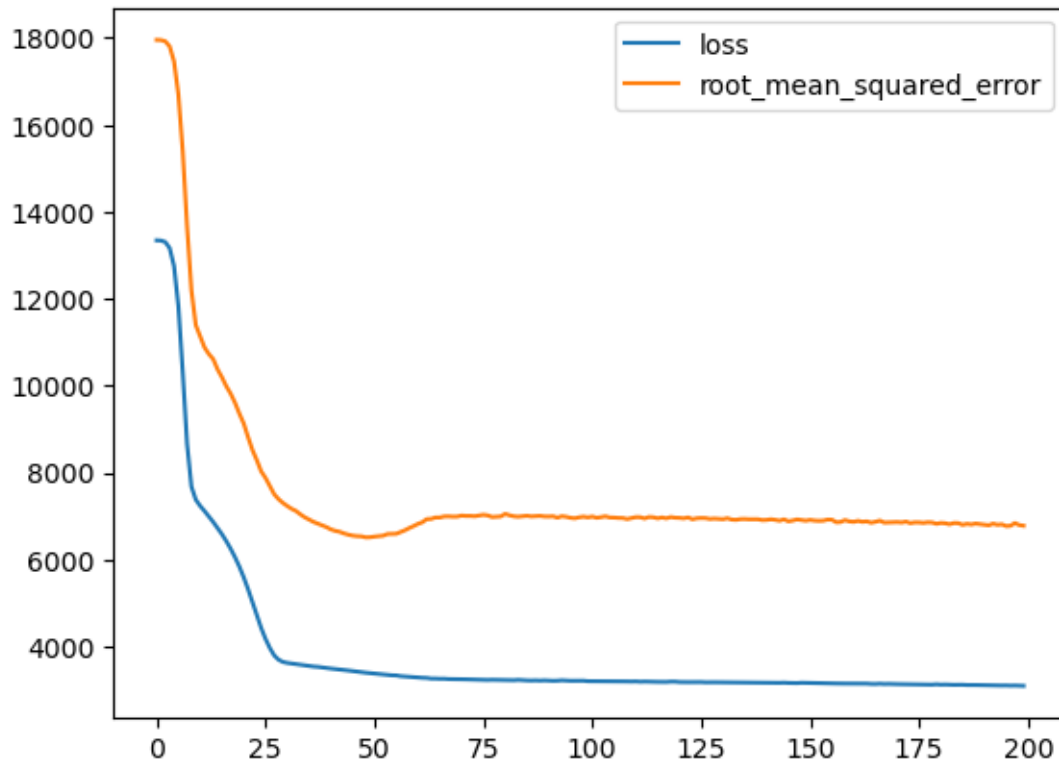
34/34          0s 1ms/step - loss:
3116.0378 - root_mean_squared_error: 6823.3462
Epoch 188/200
34/34          0s 1ms/step - loss:
3116.9180 - root_mean_squared_error: 6809.9746
Epoch 189/200
34/34          0s 1ms/step - loss:
3115.8289 - root_mean_squared_error: 6829.3032
Epoch 190/200
34/34          0s 1ms/step - loss:
3113.5586 - root_mean_squared_error: 6818.5649
Epoch 191/200
34/34          0s 1ms/step - loss:
3111.0085 - root_mean_squared_error: 6803.5649
Epoch 192/200
34/34          0s 1ms/step - loss:
3110.7161 - root_mean_squared_error: 6790.6377
Epoch 193/200
34/34          0s 1ms/step - loss:
3108.1555 - root_mean_squared_error: 6827.5752
Epoch 194/200
34/34          0s 1ms/step - loss:
3103.9084 - root_mean_squared_error: 6801.2129
Epoch 195/200
34/34          0s 1ms/step - loss:
3104.6570 - root_mean_squared_error: 6812.6372
Epoch 196/200
34/34          0s 1ms/step - loss:
3106.1416 - root_mean_squared_error: 6771.9561
Epoch 197/200
34/34          0s 1ms/step - loss:
3101.6240 - root_mean_squared_error: 6779.2866
Epoch 198/200
34/34          0s 1ms/step - loss:
3104.0681 - root_mean_squared_error: 6841.2002
Epoch 199/200
34/34          0s 1ms/step - loss:
3099.9587 - root_mean_squared_error: 6795.7041
Epoch 200/200
34/34          0s 1ms/step - loss:
3096.6313 - root_mean_squared_error: 6779.8203

```

```
[42]: <keras.src.callbacks.history.History at 0x77f8daa06950>
```

```
[43]: pd.DataFrame(model3.history.history).plot()
```

```
[43]: <Axes: >
```



Well, the model is not better in the way i wanted it to be but still better than the previous one because the RMSE is not fluctuating like model2.

And if you take a look the model is actually learning faster than model2 and more stable.

Let's evaluate.

```
[44]: model3.evaluate(X_test, y_test)
```

```
9/9          1s 38ms/step - loss:
2770.6758 - root_mean_squared_error: 6423.6587
```

```
[44]: [2770.67578125, 6423.65869140625]
```

2 Final Words

A model can be better in many ways. A better loss doesn't really mean the model is better. Always remember a model with stable results is better than a model with less error but unstable results.